ĐỀ MỞ | 40 CÂU TRẮC NGHIỆM | TƯ DUY CƠ CHẾ

ÔN TẬP CUỐI KỲ
LARGE LANGUAGE MODEL

Hệ thống kiến thức – điểm dễ hiểu sai – bài toán vận dụng – 120 câu luyện tập có giải thích (3 đề x 40 câu)

> **Triết lý ôn tập:** Không hỏi "slide ghi gì?", mà hỏi "nếu đổi một giả thiết thì điều gì còn đúng, điều gì sai, vì sao?". Tài liệu này tách rõ **kiến thức trên slide**, **suy luận trực tiếp từ cơ chế** và **câu hỏi tự biên soạn**. Không phải đề chính thức hay dự đoán chắc chắn các câu sẽ xuất hiện.

Phạm vi	7 bộ slide (96 trang) extbfvà mã nguồn các chương 2–7, Appendix E của extttLLMs-from-scratch; RLVR/GRPO notebook Chương 6 của extttreasoning-from-scratch.
Định dạng	XeLaTeX, dùng tiếng Việt Unicode, bảng tóm tắt và lời giải chi tiết.
Phù hợp	Đọc hiểu sâu trước kỳ thi; in ra để tham khảo khi thi mở tài liệu giấy.

Biên soạn độc lập từ nội dung bài giảng của TS. Nguyễn Tiến Huy và các sơ đồ trong slide. Không đại diện cho giảng viên hoặc bộ môn.

Mục lục

1 Cách đọc đề và chiến lược làm bài **3**
1.1 Bản đồ nguồn (tra ngược tài liệu gốc) . . . 3

2 Tờ công thức nhanh: dùng được trong phòng thi **3**

3 Chương 1 – Text data, sampling và embeddings **4**
3.1 Từ văn bản đến vector: ba thao tác không thể nhập làm một . . . 4
3.2 Sampling chuỗi để học next-token . . . 5
3.3 Khác nhau giữa vocabulary, context length và embedding dimension . . . 5

4 Chương 2 – Attention: từ score đến context vector **5**
4.1 Seq2seq additive attention và scaled dot-product attention . . . 5
4.2 Causal mask phải tác động trước khi chuẩn hóa . . . 6
4.3 Multi-head: phân chia kênh biểu diễn, không nhân bản dữ liệu input . . . 6
4.4 Self-attention, cross-attention, masking: chọn đúng câu hỏi . . . 6

5 Chương 3 – Kiến trúc GPT và các khối ổn định huấn luyện **6**
5.1 BERT vs GPT và mục đích của decoder-only . . . 6
5.2 Khối Transformer GPT: đọc từ đáy lên . . . 7
5.3 Logits theo vị trí, cách sinh token và đếm tham số . . . 7
5.4 Các biến chi phối shape: độ dài, từ vựng, chiều ẩn . . . 7

6 Chương 4 – Pretraining: loss, huấn luyện và decoding **7**
6.1 Train bằng teacher forcing, generate bằng autoregression . . . 8
6.2 Một epoch, một batch, backprop và tập validation . . . 8
6.3 Greedy, temperature, top-k: thứ tự thao tác . . . 8

7 Chương 5 – Fine-tuning: classification và instruction following **8**
7.1 Classification head khác LM head như thế nào? . . . 8
7.2 Instruction tuning: data format không tự sinh ra supervision mới . . . 9
7.3 Ba mask dễ bị đánh đồng . . . 9

8 Chương 6 – Alignment: RLHF, DPO, PPO và GRPO **9**
8.1 Đúng nội dung chưa chắc hợp sở thích người dùng . . . 9
8.2 RLHF truyền thống gồm ba bước khác vai trò . . . 9
8.3 DPO: vì sao cần policy và reference, nhưng không cần huấn luyện RM riêng? . . . 10
8.4 PPO vs GRPO: vai trò của value model và group . . . 10

9 Chương 7 – LLM variants: so sánh theo cơ chế, không thuộc bảng số **10**
9.1 GPT-2 qua các đời Llama: thay gì trong block? . . . 11
9.2 Llama 3.2 1B và Qwen3 0.6B: rộng và sâu là hai trục độc lập . . . 11
9.3 Các cơ chế cần diễn giải vừa đủ . . . 11

10 Phần mở rộng I – Đọc mã nguồn *LLMs-from-scratch* **12**
10.1 Bản đồ nguồn GitHub: nên đánh dấu trang khi in . . . 12
10.2 Dataset: dịch target khác dịch cửa sổ . . . 12
10.3 Các phép chiếu Q/K/V: đọc được shape và thứ tự phép toán . . . 12
10.4 Transformer block: *pre-norm*, residual và phương sai . . . 13
10.5 Decoding: ba điều dễ bị đánh tráo . . . 13
10.6 Fine-tuning phân loại: thao tác thật trên model . . . 13
10.7 Instruction collate: PAD, EOS và `-100` là ba vai trò khác nhau . . . 13
10.8 LoRA: công thức, tham số và cơ chế ban đầu . . . 14

11 Phần mở rộng II – Notebook Chương 6: RLVR và GRPO **14**
11.1 RLHF, DPO và RLVR: nguồn đánh giá đến từ đâu? . . . 15
11.2 GRPO trong notebook: 5 thao tác và 4 lỗi suy luận . . . 15
11.3 Ba bài toán vận dụng rất có thể gây nhầm . . . 16
11.4 Bảng tổng kết: hỏi “nếu thay giả thiết thì sao?” . . . 16

12 Checklist 18 ngộ nhận cần xóa trước khi vào thi **16**

13 Cách xử lý câu tính toán ngắn (không máy tính) 17

14 Đề luyện A – 40 câu trọng tâm, nhiều phương án gần đúng 18

15 Đề luyện B – 40 câu vận dụng và phản thực tế 24

16 Đề luyện C – 40 câu bám sát code Raschka và notebook RL 31

17 Đáp án và lời giải – Đề A 37

18 Đáp án và lời giải – Đề B 40

19 Đáp án và lời giải – Đề C (kiểm chứng từ mã nguồn) 42

20 Bảng ôn tập ngay trước giờ thi 44
20.1 Nếu chỉ có 15 phút: 6 câu hỏi tự vấn 44
20.2 10 dòng chốt hạ khi gặp câu hỏi bám code 44
20.3 Chiến thuật đề mở tài liệu giấy 44

1. Cách đọc đề và chiến lược làm bài

> **Thông tin giảng viên đã xác nhận:** 40 câu trắc nghiệm; đề mở, được dùng **tài liệu giấy**; có thể có tính toán nhưng không cần máy tính; mọi nội dung đã học đều có thể được kiểm tra; các điểm **dễ hiểu sai** được ưu tiên; kiểm tra tư duy và khả năng hiểu thay vì thuộc số liệu. Đây là cấu trúc thi được báo lại, **không phải tỷ trọng chương hoặc phân bố câu hỏi**.

Phương pháp loại đáp án trong 20 giây: (i) xác định *giai đoạn* (pretrain, fine-tune hay alignment); (ii) xác định *đơn vị* (token, vị trí, head, batch, số lớp hay vocabulary); (iii) xác định *hướng nhân quả* (mask thay đổi cái gì? temperature tác động lên cái gì?); (iv) kiểm tra từ khóa cực đoan: "luôn", "chắc chắn", "bắt buộc", "giống hệt", "chỉ có"; (v) so sánh *hai khả năng cùng đúng một phần* và chọn mệnh đề chính xác hơn.

Ranh giới nguồn: Các chương nền tảng đối chiếu trang slide, còn phần mở rộng ghi rõ file GitHub đã đọc. Suy luận đại số, ví dụ số học và 120 câu hỏi là *nội dung luyện tập tự xây dựng*; một số thuật ngữ nền để diễn giải hình (như teacher forcing, affine LayerNorm, attention padding mask) là *phần giải thích bổ trợ*, không phải câu chữ trích nguyên văn từ slide. Bảy slide không có mục LoRA/QLoRA riêng, nhưng **Appendix E của GitHub bổ sung LoRA**; phân biệt LoRA với QLoRA. Notebook RL bổ sung RLVR/GRPO cụ thể.

1.1 Bản đồ nguồn (tra ngược tài liệu gốc)

Ký hiệu	Tên PDF	Cụm trang quan trọng
031	`2025_Week031_TrainLLM_TokenEmbedding.pdf`	3–5 (kiến trúc), 9–10 (sampling), 12–14 (embedding + position)
032	`2025_Week032_TrainLLM_Attention.pdf`	7–8 (additive), 9–11 (scaled dot), 12–14 (self/causal/multi-head)
041	`2025_Week041-TrainLLM_Architecture.pdf`	4 (logits shape), 5–9 (các thành phần), 10–12 (GPT, sinh token)
042	`2025_Week042-TrainLLM_Pretraining.pdf`	4–6 (sinh token, loss), 7 (training), 8–10 (decoding)
051	`2025_Week051-FinetuneLLM.pdf`	3–8 (classification), 11–16 (instruction, mask loss)
052	`2025_Week052-RL.pdf`	2–7 (RLHF), 9–10 (DPO), 11 (PPO/GRPO)
062	`2025_Week062-LLM-variant.pdf`	2–3 (model variant), 6–8 (RLHF), 10–12 (DPO/PPO/GRPO)

2. Tờ công thức nhanh: dùng được trong phòng thi

> **Bảng ký hiệu:** B: batch size; T: độ dài chuỗi; S: độ dài nguồn (cross-attention); V: vocabulary size; d: embedding/model dimension; H: số attention heads; $d_h = d/H$: chiều mỗi head (khi chia đều).

Thành phần	Hình dạng và ý nghĩa
Token IDs	$[B, T]$, mỗi phần tử là **chỉ số rời rạc**, không phải vector embedding.
Token/position embedding	Hai tensor $[B, T, d]$ cộng theo phần tử $\rightarrow [B, T, d]$ (cách thể hiện trong slide GPT-2).
Query, Key, Value	Self-attention một head: $Q, K \in \mathbb{R}^{B\times T\times d_k}$, $V \in \mathbb{R}^{B\times T\times d_v}$.
Attention score	$QK^\top/\sqrt{d_k}$: $[B, T, T]$; nhiều head: $[B, H, T, T]$.
Attention output	softmax$(QK^\top/\sqrt{d_k} + M)V$; mỗi hàng trọng số cộng lại bằng 1 (trên các vị trí được phép).
Cross-attention	Query dài T, keys/values dài $S \Rightarrow$ trọng số $[B, H, T, S]$. Không phải cấu phần cross-attention của *GPT decoder-only* trong các sơ đồ đã cho.
Output logits	$[B, T, V]$ trong language modeling; lấy *vị trí cuối* để sinh token kế tiếp $\Rightarrow [B, V]$.
Next-token labels	$x = (t_0, \ldots, t_{T-1})$, $y = (t_1, \ldots, t_T)$; lệch một token, không dùng target ở tương lai làm đầu vào tại cùng vị trí.
Cross entropy LM	$\mathcal{L} = -\frac{1}{N}\sum_{i\in\mathcal{I}} \log p_\theta(y_i \mid x_{\le i})$ với $\mathcal{I}$ là tập target không bị bỏ qua.
Temperature	$p_i = \exp(z_i/\tau)/\sum_j \exp(z_j/\tau)$, $\tau > 0$. $\tau \downarrow$: phân phối nhọn hơn; sampling vẫn có thể ngẫu nhiên.
Top-k	Giữ k logits lớn nhất, đặt phần còn lại thành $-\infty$ *trước* softmax; lấy mẫu trong tập còn lại.
Tham số Q/K/V/O	Khi mỗi projection có input/output d: tổng *weight* $\approx 4d^2$, không đổi chỉ vì thay H và giữ d cố định.
Loại mask	Causal mask: *không nhìn tương lai*; attention padding mask: *không đọc pad*; label -100: *không tính loss*. Ba vai trò khác nhau.

> **Nhận diện bẫy xuyên chương:** Softmax attention chạy trên trục **keys**; softmax logits LM chạy trên **vocabulary**; softmax classification chạy trên **classes**. Cả ba đều là softmax nhưng trục và ý nghĩa xác suất hoàn toàn khác nhau.

3. Chương 1 – Text data, sampling và embeddings

[Slide: 031, tr. 7–14]

3.1 Từ văn bản đến vector: ba thao tác không thể nhập làm một

1. **Tokenization:** tách văn bản thành token theo từ vựng/tokenizer; gán *token ID* như $[2, 3, 5, 1]$. ID không chứa khoảng cách ngữ nghĩa: ID 8 không "gần" ID 9 hơn ID 200.
2. **Token embedding:** tra hàng tương ứng trong ma trận tham số $E \in \mathbb{R}^{V\times d}$; chuỗi T token biến thành T vector chiều d. Cùng ID tra cùng hàng của *một* ma trận ở một thời điểm, nhưng embedding học được có thể thay đổi qua quá trình huấn luyện.
3. **Positional embedding:** bổ sung tín hiệu vị trí. Trong kiến trúc GPT-2 minh họa trên slide, mô hình **cộng** hai vector $e(t_i) + p(i)$ để tạo biểu diễn đầu vào; không nối hai vector thành $2d$. Việc token "dog" xuất hiện hai vị trí tạo ra hai input embeddings khác nhau dù lookup token như nhau.

> **Ví dụ áp dụng.** $B = 2$, $T = 5$, $V = 100$, $d = 8$. Token IDs có shape $[2, 5]$; trọng số embedding $[100, 8]$; lookup ra $[2, 5, 8]$; cộng positional embedding ra $[2, 5, 8]$. Số token trong batch là 10, **không phải** số chiều $d = 8$.

> **Bẫy:** "Đổi ID của token nhưng giữ bảng embedding" có thể làm mô hình đổi hành vi; "đổi ID và hoán vị tương ứng các hàng của bảng embedding" thì về nguyên tắc biểu diễn tra cứu có thể giữ nguyên. *Con số ID tự nó không phải ý nghĩa ngôn ngữ.*

3.2 Sampling chuỗi để học next-token

Slide 031, trang 9–10 vẽ nhiều cửa sổ trượt từ văn bản gốc. Với một đoạn ví dụ $[a, b, c, d, e]$, một cặp độ dài 4 là input $[a, b, c, d]$ và target $[b, c, d, e]$. Mỗi vị trí đoán **token tiếp theo** bằng token hiện tại và các token trước đó. **Stride** điều khiển bước trượt cửa sổ: stride nhỏ có thể tạo nhiều mẫu chồng lắp; stride lớn giảm chồng lắp nhưng không tự biến mô hình thành bidirectional.

Nếu câu hỏi hỏi "tại sao x và y có độ dài bằng nhau mà không bị rò rỉ đáp án?": y_i khớp với vị trí *sau* x_i, còn **causal mask** cấm biểu diễn tại vị trí i chú ý đến token tương lai của chính input. [Slide: 031, tr. 9–10; 042, tr. 6]

3.3 Khác nhau giữa vocabulary, context length và embedding dimension

- V: số lựa chọn token đầu ra, quyết định kích thước *đầu từ vựng* của logits.
- T hay context length: số vị trí đang xử lý, tác động số hàng của tensor logits và kích thước attention score.
- d: độ rộng vector ẩn; tăng d thường ảnh hưởng mạnh lên tham số các phép chiếu.

Không thể kết luận mô hình có vocabulary lớn hơn thì nhất thiết có nhiều tham số hơn, hoặc context dài hơn nghĩa là số tham số nhân tỉ lệ theo chiều dài context; điều đó phụ thuộc kiến trúc và cách mã hóa vị trí.

4. Chương 2 – Attention: từ score đến context vector

[Slide: 032, tr. 4–14]

4.1 Seq2seq additive attention và scaled dot-product attention

Additive attention trong sơ đồ encoder–decoder dùng *trạng thái decoder* s_{t-1} và *các hidden states encoder* h_i để tính score, sau đó softmax qua **các vị trí nguồn**:

$$e_{t,i} = v_a^\top \tanh\big(W_a[s_{t-1}; h_i]\big), \quad \alpha_{t,i} = \frac{e^{e_{t,i}}}{\sum_j e^{e_{t,j}}}, \quad c_t = \sum_i \alpha_{t,i} h_i.$$

Sơ đồ trang 7–8: *ở mỗi* bước decoder, trọng số attention thay đổi theo trạng thái decoder mới; **context vector không phải là một vector cố định cho cả câu nguồn**. [Slide: 032, tr. 7–8]

Scaled dot-product attention trong Transformer:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right) V.$$

$QK^\top$ cho *độ tương hợp*, softmax cho *tỷ trọng*, phép nhân V cho *nội dung tổng hợp*. Query/Key/Value là **vai trò**, không nhất thiết từ ba tensor dữ liệu gốc khác nhau; với self-attention, các phép chiếu có thể nhận cùng X.

> **Sai lầm rất thường gặp:** Thay giá trị V (giữ Q, K) không làm đổi ma trận *attention weights*, nhưng làm đổi *output*; đổi Q hoặc K (giữ V) có thể làm đổi weights và output. $\sqrt{d_k}$ dùng

> **chiều key của một head**, không phải số token T và không phải vocabulary V.

4.2 Causal mask phải tác động trước khi chuẩn hóa

Với token ở vị trí i, các key ở vị trí $j > i$ bị gán $-\infty$ trong score, nên $e^{-\infty} = 0$ và tổng softmax chuẩn hóa lại trên các key *được phép*. Ví dụ score trước mask là $[1, 2, 3, 4]$, token query tại vị trí 2 (đếm từ 1) chỉ được đọc vị trí 1 và 2: $[1, 2, -\infty, -\infty]$. Hai trọng số đầu được chuẩn hóa để tổng bằng 1; **không** lấy softmax bốn giá trị rồi chỉ gán hai xác suất cuối bằng 0 mà bỏ qua bước chuẩn hóa lại. [Slide: 032, tr. 9, 13]

Sơ đồ "Casual attention" trong slide trang 13 thực chất minh họa *causal attention*. Causal mask không làm câu trở thành *causal reasoning*; đây là giới hạn thông tin theo thời gian sinh token.

4.3 Multi-head: phân chia kênh biểu diễn, không nhân bản dữ liệu input

Với $X \in \mathbb{R}^{B\times T\times d}$ và H heads chia đều, một cách cài đặt chuẩn là chiếu Q, K, V thành $[B, T, d]$, reshape thành $[B, H, T, d_h]$, tính H ma trận $[T, T]$, rồi concat kết quả và output projection về $[B, T, d]$. **Các head có trọng số projection học được khác nhau**; số head tăng không mặc nhiên làm dimension cuối hay số lượng tham số tăng nếu d giữ nguyên. [Slide: 032, tr. 12, 14]

> **Bài toán shape.** Input $[2, 4, 8]$, $H = 2$, $d_h = 4$. Q sau chia head có shape $[2, 2, 4, 4]$; attention scores $[2, 2, 4, 4]$; output mỗi head $[2, 2, 4, 4]$; ghép head $[2, 4, 8]$. Nếu một câu đưa $[2, 4, 2, 4]$ cho score, họ đang nhầm trục *query positions* với trục *channels*.

4.4 Self-attention, cross-attention, masking: chọn đúng câu hỏi

Dạng	Query đến từ	Key/Value và điểm cần nhớ
Self-attention	Chuỗi đang xử lý	Chính chuỗi đó; W_Q, W_K, W_V vẫn có thể khác nhau.
Causal self-attention	Chuỗi đang xử lý	Chính chuỗi đó *nhưng* cấm key tương lai; dùng trong GPT decoder-only.
Cross-attention	Một chuỗi (thường decoder)	Một chuỗi khác (thường encoder); scores $T \times S$.
Attention padding mask	Bất kỳ dạng attention thích hợp	Loại bỏ key là padding; độc lập với việc có causal mask hay không.

> **Bẫy "tính hiệu quả":** Nhìn vào tensor $T \times T$ có thể suy ra tính toán attention toàn phần và bộ nhớ score tăng bậc hai theo T (giữ các chiều khác), nhưng **không được suy ra** tổng thời gian suy luận luôn chính xác tỉ lệ T^2 trên mọi phần cứng/kiến trúc.

5. Chương 3 – Kiến trúc GPT và các khối ổn định huấn luyện

[Slide: 031, tr. 3–5; 041, tr. 3–12]

5.1 BERT vs GPT và mục đích của decoder-only

Trong slide 031 trang 3, **BERT** đại diện cách xử lý *encoder*/masked-word, có thể nhìn ngữ cảnh hai phía trong dạng nhiệm vụ minh họa; **GPT** là *decoder-only autoregressive*, học dự đoán token kế tiếp. Không suy ra "GPT không hiểu ngữ cảnh" hay "BERT không bao giờ dùng được để phân

loại"; khác biệt trọng tâm là **kiểu attention, mục tiêu huấn luyện và cách sử dụng**. Đầu ra GPT là logits cho các vị trí và token vocabulary, rồi chọn token tiếp theo để nối vào prompt. [Slide: 031, tr. 3–4; 041, tr. 12]

5.2 Khối Transformer GPT: đọc từ đáy lên

Các sơ đồ 041 trang 9–10 cho dạng *pre-norm*:

$$U = X + \text{Dropout}\big(\text{MHA}(\text{LN}_1(X))\big), \quad Y = U + \text{Dropout}\big(\text{FFN}(\text{LN}_2(U))\big).$$

Bỏ dropout ở inference (mô hình ở chế độ đánh giá). Cả hai residual additions giữ kích thước tensor khi cùng d. Một block *không phải* toàn bộ GPT; nhiều blocks xếp chồng trước final norm và output layer. [Slide: 041, tr. 8–11]

- **LayerNorm:** chuẩn hóa kích hoạt để hỗ trợ ổn định học. Chuẩn hóa theo các đặc trưng của mẫu/token trong cấu hình minh họa, *không bắt buộc* lấy thống kê theo batch như BatchNorm. Affine γ, β sau chuẩn hóa có thể khiến output cuối không còn mean đúng 0/variance đúng 1.
- **GELU:** phi tuyến trơn được so với ReLU trong slide trang 6; **không** có nghĩa GELU luôn chính xác hoặc hội tụ nhanh hơn ReLU ở mọi bài toán.
- **Residual shortcut:** tạo đường truyền tín hiệu/gradient trực tiếp, giảm khó khăn vanishing gradient; không phải đảm bảo hết mọi vấn đề tối ưu.
- **Dropout:** regularization, ngẫu nhiên bỏ kích hoạt ở *training*; khi evaluation/inference chuẩn thông thường sẽ tắt dropout và không chủ động làm output ngẫu nhiên theo dropout.

> **Bẫy phân biệt:** *LayerNorm không tương đương Dropout*; residual không phải phép cộng embedding vị trí; dropout không phải causal mask. Câu nói "bỏ residual thì shape đổi" thường sai nếu chỉ xét chiều tensor: shape có thể giữ nguyên, nhưng đường truyền gradient bị ảnh hưởng.

5.3 Logits theo vị trí, cách sinh token và đếm tham số

Với GPT nhận T token, logits toàn bộ chuỗi có shape $[B, T, V]$; khi generate, dùng logits ở *vị trí cuối của prompt đang có* để dự đoán một token kế tiếp, rồi thêm token vào chuỗi và tiếp tục. Softmax logits LM được thực hiện *qua vocabulary*. Dự đoán bằng argmax là greedy; không đồng nghĩa với đào tạo mô hình. [Slide: 041, tr. 4, 12]

Trong MHA cơ bản có W_Q, W_K, W_V, W_O kích thước $d \times d$; phần *weight matrices* xấp xỉ $4d^2$. Với $d = 8$: $4 \cdot 8^2 = 256$ trọng số, **không tính bias**, norm, FFN, embeddings. Nếu đổi $H = 2$ lên $H = 4$ với tổng d giữ nguyên và cấu trúc projection giữ nguyên, không được nhân gấp đôi con số này. [Slide: 032, tr. 14; 041, tr. 9–11]

5.4 Các biến chi phối shape: độ dài, từ vựng, chiều ẩn

> **Phân tích phản thực tế.** Tăng T khi d, V cố định: logits tăng số *vị trí*; attention score tăng số *cặp vị trí*; thông thường **không cần** thay tensor trọng số projection Q/K/V. Tăng V (giữ d): thay size embedding table và output vocabulary head. Tăng d: đổi độ rộng biểu diễn và nhiều ma trận trọng số trong mô hình.

6. Chương 4 – Pretraining: loss, huấn luyện và decoding

[Slide: 042, tr. 2–10]

6.1 Train bằng teacher forcing, generate bằng autoregression

Trong training, input và target lệch nhau một token; mô hình có thể tính loss tại *nhiều vị trí song song* nhờ causal mask. Giá trị **target đúng** để tính loss không phải token model đã đoán tại vị trí trước. **Trong generation**, mô hình nhận chuỗi hiện có, dự đoán token tiếp theo và nối token đó vào prompt để bước sau sử dụng. Đây là khác biệt *chế độ sử dụng* của cùng nguyên tắc next-token. [Slide: 042, tr. 4–7]

Cross entropy trên các target hợp lệ $-\log p(y_i \mid x_{\le i})$: đặt xác suất lớn vào token đúng thì loss nhỏ. Không thể kết luận "sinh đúng bằng greedy trên một ví dụ" đồng nghĩa loss training bằng 0; loss phụ thuộc phân phối xác suất *của tất cả target tính vào trung bình*.

> **Câu tính không cần máy tính.** Hai phân phối dự đoán tại token đúng lần lượt có xác suất 0.8 và 0.2. Loss của trường hợp đầu *thấp hơn* vì $-\log 0.8 < -\log 0.2$. Không cần tính số thập phân. Nếu y bị gán -100 ở cơ chế `ignore_index=-100`, vị trí ấy *không góp loss*, không có nghĩa target token ID thật là -100.

6.2 Một epoch, một batch, backprop và tập validation

Slide 042 trang 7 liệt kê vòng lặp: epoch → các mini-batch → reset gradients → forward/loss → backward → update weights. *Một epoch* là đi qua tập train một lượt, không phải một lần update; *batch size* xác định số ví dụ mỗi update (trừ accumulation). Evaluation trên validation không được dùng như các batch huấn luyện để cập nhật trọng số trong workflow tiêu chuẩn minh họa.

6.3 Greedy, temperature, top-k: thứ tự thao tác

- **Greedy:** chọn argmax, deterministic khi logits và tie-breaking xác định.
- **Temperature $\tau > 0$:** chia logits cho τ rồi softmax. $\tau < 1$ làm phân phối sắc hơn, $\tau > 1$ phẳng hơn; với τ dương, **không làm thay đổi vị trí argmax** của logits (trừ các trường hợp xử lý số/tie). Sampling mới quyết định có ngẫu nhiên hay không.
- **Top-k:** chỉ cho phép lấy mẫu trong k token có logits cao nhất; xác suất các token khác bằng 0 sau mask và chuẩn hóa lại. Khi $k = 1$, tương đương chọn token top-1 (bỏ tie ambiguity); khi $k = V$, không cắt bớt token nào.
- **Ghép cả hai**: bước lọc top-k và temperature có thể phối hợp. Không được nhầm k là *số token được sinh* hay *độ dài context*.

[Slide: 042, tr. 8–10]

> **Bẫy:** "Hạ temperature làm greedy đổi token" là sai trong điều kiện logits cố định, $\tau > 0$. "Top-k bảo đảm văn bản đa dạng hơn greedy" cũng không đúng nếu $k = 1$; với $k > 1$, chỉ *tạo khả năng* lấy mẫu đa dạng.

7. Chương 5 – Fine-tuning: classification và instruction following

[Slide: 051, tr. 2–16]

7.1 Classification head khác LM head như thế nào?

Ở slide 051 trang 6, thay tầng *vocabulary projection* (ví dụ $768 \to 50{,}257$) thành *class head* ($768 \to 2$ cho spam/not spam). Khi dùng *classification-finetuned GPT* đúng thiết kế, input có thể chỉ là

email mà không cần prompt dài yêu cầu trả lời "yes/no"; logits cuối là **điểm các lớp**, không phải điểm xác suất toàn bộ token trong vocabulary. [Slide: 051, tr. 3–7]

Vị trí dùng cho phân loại: slide trang 7 minh họa dùng representation của token cuối cùng. Không suy ra đây là *quy tắc duy nhất cho mọi classifier*; cũng không được lấy *mọi* token ID của LM làm một nhãn phân loại. Một classifier 2 lớp có thể có loss cross entropy, nhưng trục class size là 2, không phải V.

7.2 Instruction tuning: data format không tự sinh ra supervision mới

Ví dụ dữ liệu gồm `instruction`, `input`, `output`. Cùng một cặp yêu cầu–câu trả lời có thể được format dưới *Alpaca* hay *Phi-3* như slide 051 trang 13; khác template không đồng nghĩa khác bài toán hoặc khác nhãn. Khi huấn luyện mô hình để trả lời instruction, thông thường văn bản được ghép, tokenize, pad/truncate và xây target shift đúng. [Slide: 051, tr. 11–14, 16]

7.3 Ba mask dễ bị đánh đồng

1. **Causal attention mask:** chặn nhìn token tương lai trong tính hidden state.
2. **Padding attention mask:** chặn attention tới padding ở những cài đặt có dùng loại mask này.
3. **Loss mask bằng -100:** chỉ thị cho cross entropy *bỏ qua* vị trí target (pad hoặc instruction, theo chính sách huấn luyện).

Trên slide 051 trang 14–15, `-100` dùng cho target padding và *tuỳ chọn* cho instruction tokens. **Không cần thay input token IDs thành** -100; thay đổi nằm ở *labels dùng cho loss*. Chỉ mask pad mà không mask instruction là một lựa chọn hợp lệ; nếu mask instruction, gradient loss tập trung vào response. Chú ý cơ chế shift để xác định *vị trí label nào* thực sự có loss. [Slide: 051, tr. 14–15]

> **Phản thực tế.** Hai cách huấn luyện trên cùng prompt–answer: (A) loss trên cả instruction lẫn answer; (B) loss chỉ trên answer. Cả hai đều có thể chạy; B ưu tiên trực tiếp học sinh phần trả lời hơn. Không đúng nếu nói A "vi phạm causal attention" hay B "xóa instruction khỏi input".

8. Chương 6 – Alignment: RLHF, DPO, PPO và GRPO

[Slide: 052, tr. 2–11; 062, tr. 4–12]

8.1 Đúng nội dung chưa chắc hợp sở thích người dùng

Slide cho ví dụ cùng câu hỏi tư vấn laptop nhưng có *technical answer* và *user-friendly answer*. Cả hai có thể đúng về mặt nội dung, song khác độ hữu ích với đối tượng người dùng. Đây là động lực cho **preference alignment**. Không có nghĩa preference optimization thay thế hoàn toàn factual correctness hay pretraining.

8.2 RLHF truyền thống gồm ba bước khác vai trò

1. **SFT (supervised fine-tuning):** human-written desirable responses giúp có base policy biết trả lời chỉ dẫn.
2. **Reward model (RM):** lấy nhiều ứng viên cho cùng prompt, con người *xếp hạng/chọn ưu tiên*; train một model học ánh xạ output tới mức ưa thích tương đối.
3. **RL policy optimization:** policy sinh output, RM cung cấp reward, thuật toán PPO (như hình) cập nhật policy để cải thiện reward trong khi có cơ chế điều tiết độ lệch so với reference policy

(sơ đồ thể hiện KL).

[Slide: 052, tr. 5–7; 062, tr. 6–8]

Bẫy: RM *không phải* "bộ phán định chân lý tuyệt đối". RM học từ preferences được thu thập, có sai lệch/giới hạn; chính sách tối ưu RM không đồng nghĩa luôn tối ưu sự hữu ích thực tế. Thứ tự SFT → RM → policy optimization trong *hình RLHF truyền thống* là điều cần nhớ, không nhầm với DPO.

8.3 DPO: vì sao cần policy và reference, nhưng không cần huấn luyện RM riêng?

Slide 052 trang 10 ghi trực tiếp:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x,y_w,y_l)\sim D}\left[\log \sigma\left(\beta \log \frac{\pi_\theta(y_w \mid x)}{\pi_{\text{ref}}(y_w \mid x)} - \beta \log \frac{\pi_\theta(y_l \mid x)}{\pi_{\text{ref}}(y_l \mid x)}\right)\right].$$

- x: prompt; y_w: chosen/winner; y_l: rejected/loser; π_θ: policy đang cập nhật; π_{ref}: reference policy cố định trong thiết lập thông dụng; $\beta > 0$: hệ số của preference margin.
- DPO dùng **dữ liệu cặp preference** để tối ưu trực tiếp policy, *không cần bước huấn luyện explicit reward model và PPO như sơ đồ RLHF*. Nhưng vẫn có *reference*, không phải "bỏ luôn model gốc".
- Hệ số bên trong σ là **chênh lệch log-likelihood ratio so với reference**, không đơn giản là $\log \pi_\theta(y_w) - \log \pi_\theta(y_l)$.

Đọc dấu của margin, không cần máy tính. Giả sử $\pi_\theta(y_w)/\pi_{\text{ref}}(y_w) = 2$ và $\pi_\theta(y_l)/\pi_{\text{ref}}(y_l) = 1/2$. Khi đó margin $= \beta[\log 2 - \log(1/2)] = \beta \log 4 > 0$, nên $\sigma(\text{margin}) > 1/2$. Nếu đổi hai tỷ số thì margin âm và loss lớn hơn. Đây là *minh họa algebra*, không phải ví dụ dữ liệu gốc của slide.

Bẫy DPO: Policy có thể làm tăng xác suất cho *cả hai* câu trả lời và vẫn thay đổi rất ít preference margin nếu các tỷ số tăng tương ứng. DPO tối ưu *tương quan* chosen–rejected **theo reference**, không đòi hỏi $p(y_w)$ nhất thiết lớn hơn $p(y_l)$ về trị tuyệt đối trong mọi tình huống. β cũng *không* phải là learning rate.

8.4 PPO vs GRPO: vai trò của value model và group

Trong sơ đồ 052 trang 11, **PPO** dùng *value model* để ước lượng nền cho lợi thế A (hình nhắc GAE) cùng reward và reference/KL. **GRPO** sinh *nhiều output cho cùng prompt*, lấy rewards $r_1, \ldots, r_G$ và tính *group-relative advantages*; sơ đồ không có một value model riêng như PPO. **Không đồng nghĩa GRPO không dùng reward** hoặc luôn bỏ qua reference/KL. [Slide: 052, tr. 11; 062, tr. 12]

Bài toán xếp hạng tương đối. Nếu ba responses cùng prompt có reward $(1, 3, 5)$, mức thưởng ở giữa không phải "tốt tuyệt đối"; nó đúng *mốc trung bình của nhóm*. Câu hỏi gài bẫy thường cố đánh đồng raw reward với advantage tương đối. Slide thể hiện group computation, còn phép chuẩn hóa số học cụ thể tùy cài đặt.

9. Chương 7 – LLM variants: so sánh theo cơ chế, không thuộc bảng số

[Slide: 062, tr. 2–3]

9.1 GPT-2 qua các đời Llama: thay gì trong block?

Sơ đồ "GPT to Llama" đối chiếu GPT-2 XL với Llama 2 7B, Llama 3 8B, Llama 3.1 8B và Llama 3.2 1B. Có thể quan sát các thay đổi *trên chính sơ đồ*: LayerNorm so với RMSNorm; GELU so với cấu trúc FFN có SiLU/gating (SwiGLU); positional embedding tuyệt đối so với RoPE/RoPE rescaling; attention thường so với grouped-query attention ở một số biến thể; vocabulary/context/độ sâu/rộng thay đổi. **Không được suy ra** mọi Llama cùng cấu hình, hoặc phiên bản mới luôn có nhiều parameters hơn.

9.2 Llama 3.2 1B và Qwen3 0.6B: rộng và sâu là hai trục độc lập

Theo *sơ đồ cụ thể ở* slide trang 3: Llama 3.2 1B có **16 blocks**, embedding dimension **2048**, 32 heads; Qwen3 0.6B có **28 blocks**, embedding dimension **1024**, 16 heads. Cả hai thể hiện RMSNorm, grouped-query attention, RoPE và FFN dùng SiLU/gating; sơ đồ Qwen thêm *Q/K RMSNorm*. Đây là ví dụ rõ về **rộng hơn nhưng nông hơn** đối lập **sâu hơn nhưng hẹp hơn**. [Slide: 062, tr. 3]

> **Bẫy tổng quát hóa:** "Qwen có ít parameters hơn nên phải nông hơn" sai trong cặp được nêu. "Head nhiều hơn nghĩa là layer sâu hơn" sai vì heads và layers là hai trục. "Context window dài hơn chắc chắn chất lượng mọi bài toán tốt hơn" cũng không được slide chứng minh.

9.3 Các cơ chế cần diễn giải vừa đủ

- **RMSNorm** khác LayerNorm ở cách chuẩn hóa; sơ đồ cho biết *có thay đổi normalization*, không phải chứng minh RMSNorm luôn tốt hơn.
- **RoPE** biểu diễn thông tin vị trí trong attention, khác cách *cộng learned positional embedding* của sơ đồ GPT-2. Không suy ra bỏ hết thông tin vị trí.
- **Grouped-query attention (GQA)** cho phép nhiều query heads chia sẻ một tập key/value heads nhỏ hơn; không đồng nghĩa "chỉ một head attention".
- **Weight tying** chia sẻ/trói trọng số giữa token embedding và output projection khi shape tương thích; giảm số ma trận tham số độc lập, *không* loại bỏ output vocabulary logits.

Các định nghĩa ngắn ở đây là diễn giải cơ chế để đọc sơ đồ, không phải nội dung mô tả chi tiết được viết thành đoạn trong slide.

10. Phần mở rộng I – Đọc mã nguồn *LLMs-from-scratch*

Vì sao cần đọc code? Bảy bộ slide xây dựng trực giác; mã nguồn cho biết chính xác *tensor nào* được tạo, *chiều nào* được rút gọn, *tham số nào* được cập nhật và *lệnh nào* tác động đến gradient. Phần này đối chiếu trực tiếp với các chương 2–7 và Appendix E của Sebastian Raschka. Các câu hỏi và ví dụ số bên dưới do tài liệu này tự biên soạn, **không phải trích nguyên câu hỏi của tác giả**. [Code: R2–R7, RE]

10.1 Bản đồ nguồn GitHub: nên đánh dấu trang khi in

Đối chiếu nhánh main ngày 25/10/2026: LLMs-from-scratch `cdbd33e6d57a`*; reasoning-from-scratch* `a788466dc85c`*. Khi GitHub thay đổi về sau, dùng mã commit này để tra bản gốc.*

Ký hiệu	File / chương	Nội dung cần tra cứu
R2	`ch04/.../gpt.py`, mục Ch. 2	`GPTDatasetV1`: cửa sổ có stride, input/target dịch 1 vị trí; tokenizer GPT-2.
R3	`gpt.py`, mục Ch. 3	`MultiHeadAttention`: $Q/K/V$, reshape, transpose, causal mask, softmax, dropout, output projection.
R4	`gpt.py`, mục Ch. 4	`LayerNorm`, `FeedForward`, `TransformerBlock`, `GPTModel`, sinh greedy.
R5	`Code generation`	Dùng logits vị trí cuối, cắt context, top-k, temperature, sampling, EOS.
R6	`Code classification`	Chuyển LM head thành class head; last-token logits; đóng băng và mở khóa tham số.
R7	`Code instruction`	Instruction formatting, padding, target shift, `ignore_index=-100`.
RE	`appendix-E.ipynb`	LoRA: low-rank matrices, freeze base, scaling α/r, trainable parameters.
RM6	`Ch. 6 RL notebook`	RLVR và phiên bản GRPO tối giản: reward, rollout, advantage, sequence log-prob và policy-gradient.

10.2 Dataset: dịch target khác dịch cửa sổ

Trong `GPTDatasetV1`, tác giả tokenize toàn bộ đoạn văn, rồi với mỗi chỉ số bắt đầu i lấy $x = \text{ids}[i : i+L]$ và $y = \text{ids}[i+1 : i+L+1]$. Biến `stride` chỉ quyết định **khoảng cách giữa hai cửa sổ**, còn *target luôn dịch đúng một token* so với input, bất kể stride bao nhiêu. Vòng lặp trong phiên bản được kiểm tra sử dụng các điểm bắt đầu thuộc `range(0, len(ids)-L, stride)`. [Code: R2]

Hai bước tính. Với 12 token, $L = 4$, stride $= 3$, chỉ số mở đầu là $0, 3, 6$ **(3 samples)**. Cửa sổ đầu dùng tokens $0, 1, 2, 3$ làm input, $1, 2, 3, 4$ làm target. Không nhầm "dịch 1 token trong cặp (x, y)" với "dịch 3 tokens giữa hai mẫu". Không phải mọi cửa sổ hợp lý về mặt toán học đều nhất thiết được tạo: phải đọc *range* thật sự trong code.

10.3 Các phép chiếu Q/K/V: đọc được shape và thứ tự phép toán

`MultiHeadAttention` chiếu cùng input $[B, T, d_{in}]$ qua ba linear *học tham số riêng*; khi $d_{out} = Hd_h$, các tensor được reshape $[B, T, H, d_h]$, transpose thành $[B, H, T, d_h]$, rồi tạo scores $[B, H, T, T]$. Một `assert` yêu cầu d_{out} chia hết cho số heads. Upper-triangular causal mask chứa các ô *phía trên đường chéo*, gán chúng thành $-\infty$ **trước softmax**. [Code: R3]

Bẫy code thật: Trong ví dụ Raschka, *attention dropout được áp sau softmax*. Vì vậy, trước dropout, mỗi hàng trên các vị trí hợp lệ cộng lại thành 1; **trong lúc training, sau dropout, tổng hàng không nhất thiết bằng 1**. `qkv_bias=False` chỉ bỏ *bias* của Q/K/V; các ma trận trọng số Q/K/V vẫn được học.

10.4 Transformer block: *pre-norm*, residual và phương sai

Đây là một biến thể GPT minh họa: `shortcut=x` → `norm1` → attention → dropout → cộng shortcut; sau đó lặp lại với `norm2` và feed-forward. Về đại số, $x' = x + \text{Drop}(\text{Attn}(\text{LN}(x)))$; $x'' = x' + \text{Drop}(\text{FFN}(\text{LN}(x')))$. `LayerNorm` tính trung bình và phương sai theo **chiều cuối của tensor**, có ϵ và hai tham số affine, dùng `unbiased=False` trong cách tính variance. [Code: R4]

Đọc suy luận shape: Token lookup $[B, T, d]$, position lookup $[T, d]$ được broadcast cộng vào từng batch, block giữ $[B, T, d]$, LM head tạo logits $[B, T, V]$. Khi *generate*, chỉ lấy `logits[:, -1, :]` để dự đoán token kế tiếp; điều này **không có nghĩa** pretraining chỉ tính loss ở vị trí cuối. [Code: R4–R5]

10.5 Decoding: ba điều dễ bị đánh tráo

`generate` trong code Chương 5: cắt context còn `context_size`, lấy logits cuối; nếu có top-k thì gán logits nằm ngoài ngưỡng top-k thành $-\infty$; với $\tau > 0$, chia logits cho τ và *sample*, còn $\tau = 0$ theo nhánh code là *argmax*. Có thể dừng sớm theo EOS trong hàm này. [Code: R5]

Ba kết luận phản ví dụ: (1) τ càng lớn thì phân phối *có xu hướng* phẳng hơn, không đổi thứ hạng logits khi $\tau > 0$; (2) top-k chỉ giới hạn tập ứng viên, **không bắt buộc** model chọn token lớn nhất trừ trường hợp $k = 1$; (3) `context_size` giới hạn *đầu vào mỗi bước forward*, không đồng nghĩa tổng số token đã trả về luôn phải bé hơn context size.

10.6 Fine-tuning phân loại: thao tác thật trên model

Ở Chương 6, sau khi đóng băng backbone, code thay LM output head bằng head $d \to C$ ($C = 2$ ở bài spam). Đồng thời mở khóa *transformer block cuối* và *final normalization* để train cùng classification head. Loss được lấy từ **logits ở vị trí cuối**, `model(inputs)[:, -1, :]`, đối chiếu với nhãn toàn câu. Khi inputs đã được padding đến cùng chiều dài, đây là **vị trí tensor cuối**, *có thể là vị trí PAD*, không phải mặc nhiên token có nghĩa cuối cùng. [Code: R6]

Bài tập so sánh: Với $B = 4, T = 10, C = 2$, output model sau thay head có shape $[4, 10, 2]$, nhưng logits đưa vào cross entropy cho bài toán phân loại câu là $[4, 2]$. Nếu câu hỏi nói "mọi token đều có nhãn spam/ham trong loss ở đây", đó là nhầm bài toán *sequence labeling* với *sequence classification*.

10.7 Instruction collate: PAD, EOS và `-100` là ba vai trò khác nhau

Trong `custom_collate_fn`, mỗi example được nối thêm một token EOS/PAD ID (ví dụ GPT-2 là 50256), sau đó pad đủ chiều dài batch; inputs lấy chuỗi trừ token cuối, targets lấy chuỗi trừ token đầu. *Chỉ các vị trí padding từ lần xuất hiện thứ hai trở đi trong targets* được thay bằng `-100`. Đây là *ignore index cho hàm loss*, không phải token ID hợp lệ để model học sinh ra. [Code: R7]

Phân biệt ba tác vụ mask: *Causal attention mask* ngăn một query nhìn token tương lai; *attention padding mask* ngăn attention đến PAD nếu được cài; còn `target=-100` bảo hàm loss bỏ qua vị trí. **Mask loss không tự xóa PAD khỏi quá trình forward.** Code Chương 7 cơ bản còn tính loss trên cả phần instruction/prompt; *mask prompt tokens* là một lựa chọn/biến thể, không mặc định xảy ra chỉ vì dữ liệu định dạng instruction–response.

10.8 LoRA: công thức, tham số và cơ chế ban đầu

Appendix E không cập nhật ma trận nền đã học W; thay vào đó, thêm nhánh điều chỉnh hạng thấp. Theo quy ước nhân từ bên trái trong code (x ở dạng hàng), đầu vào d_{in}, đầu ra d_{out}, $A \in \mathbb{R}^{d_{in}\times r}$, $B \in \mathbb{R}^{r\times d_{out}}$:

$$y = \text{Linear}_{\text{frozen}}(x) + \frac{\alpha}{r}\, xAB, \qquad N_{\text{LoRA}} = r(d_{in} + d_{out}).$$

`LoRALayer` khởi tạo A khác 0, $B = 0$, nên *lúc bắt đầu* nhánh LoRA cho ra 0; model hoạt động như bản pretrained (khi không xét khác biệt train/eval). `LinearWithLoRA` cộng kết quả của linear gốc và LoRA. Code đóng băng toàn bộ model trước, sau đó thay thế các `Linear` bằng adapter có tham số mới cần train. [Code: RE]

Hai câu tính nhanh: Nếu $d_{in} = 12, d_{out} = 8, r = 2$, LoRA có $2(12 + 8) = 40$ tham số, thay vì $12 \cdot 8 = 96$ tham số weight gốc. Nếu tăng r từ 2 lên 4 khi α không đổi, vừa tăng số tham số lên gấp đôi, vừa đổi hệ số α/r từ $\alpha/2$ thành $\alpha/4$; không được nói rằng mọi đầu ra hoặc lợi ích chắc chắn tăng gấp đôi.

Giới hạn nguồn: Appendix E bàn về **LoRA**. *QLoRA* (lượng tử hóa base model và huấn luyện adapter) là khái niệm liên quan, nhưng **không được giải thích thành một pipeline QLoRA đầy đủ trong những file chính đã đối chiếu**; đừng đánh đồng "low-rank" với "quantization". Đây là ghi chú đối chiếu, không quy phần kiến thức QLoRA cho Appendix E.

11. Phần mở rộng II – Notebook Chương 6: RLVR và GRPO

Nguồn trực tiếp: `reasoning-from-scratch/ch06/01_main-chapter-code/ch06_main.ipynb`, các mục 6.1–6.10 và 6.13. Đây là phần *RL training cho reasoning* dựa trên reward có thể kiểm chứng, **không phải** chỉ lặp lại slide về human preferences.

11.1 RLHF, DPO và RLVR: nguồn đánh giá đến từ đâu?

Phương pháp	Tín hiệu học	Điều không được suy ra
SFT	Cặp prompt–đáp án mẫu	Không trực tiếp tối ưu sở thích tương đối nếu không có nhãn preference.
RLHF kiểu PPO	Reward model học từ xếp hạng của người; policy tối ưu reward	Reward model không phải bộ kiểm chứng chân lý tuyệt đối.
DPO	Cặp *chosen/rejected* và reference policy, loss trực tiếp	Không cần *huấn luyện riêng* RM/PPO trong pipeline DPO chuẩn; vẫn cần preference data.
RLVR + GRPO trong notebook	*Reward tự kiểm chứng* từ tính đúng và điều kiện định dạng, sau đó so sánh các rollout cùng prompt	Không cần xếp hạng thủ công từng cặp ở bước tính reward; không có nghĩa mọi nhiệm vụ đều tự kiểm chứng được.

RL gắn với giai đoạn nào? Pretraining chủ yếu tối ưu next-token objective; RL có thể được thực hiện ở post-training để cải thiện chất lượng câu trả lời theo mục tiêu cấp chuỗi. Notebook phân biệt *training-time scaling* (thêm compute để train) với *inference-time scaling* (dùng thêm compute khi trả lời); hai hướng có thể bổ sung cho nhau. [Code: RM6, 6.1]

11.2 GRPO trong notebook: 5 thao tác và 4 lỗi suy luận

1. **Sample rollouts:** từ *cùng một prompt* tạo nhiều đáp án $y^{(i)}$. Notebook dùng temperature và top-p sampling; hàm sample được chạy với `no_grad`. Đó là sinh mẫu, **không** phải đường truyền gradient qua các quyết định token rời rạc.
2. **Reward:** hàm `reward_rlvr` kiểm tra câu trả lời cuối bằng bộ kiểm chứng toán. Ví dụ chỉ nhận điểm 1 nếu đáp án *đúng và được trích ra theo định dạng* `\boxed{...}`; nội dung số đúng nhưng không theo định dạng này có thể bị 0.
3. **Advantage tương đối:** một nhóm phần thưởng $r_1, \dots, r_G$ tạo ra
$$A_i = \frac{r_i - \bar{r}}{s_r + \epsilon}.$$
Notebook dùng `torch.std()` (mặc định sample standard deviation với correction 1 trong PyTorch), không chỉ trừ raw reward. Với $[1, 1, 0, 0]$ thì advantages có dấu $[+, +, -, -]$ (giá trị demo khoảng ± 0.866), **không phải** $[1, 1, 0, 0]$.
4. **Tính log-prob của đáp án đã lấy mẫu:** model forward lại với gradient, gather log-prob của *các token phản hồi* (không tính loss cho prompt), rồi *cộng* qua chuỗi:
$$\log \pi_\theta(y \mid x) = \sum_{t=1}^{T_y} \log \pi_\theta(y_t \mid x, y_{<t}).$$
Trung bình per token được notebook sử dụng để *so điểm chuỗi không cùng độ dài* ở ví dụ trước đó; cho policy-gradient loss trong phần này thì dùng tổng.
5. **Policy-gradient loss tối giản:**
$$\mathcal{L}_{\text{PG}} = -\frac{1}{G} \sum_{i=1}^{G} A_i \log \pi_\theta(y^{(i)} \mid x).$$
Trong code có `advantages.detach()` để xem A là tín hiệu cố định, **không** cho gradient chạy ngược qua reward/normalization. Dấu trừ vì optimizer tối thiểu hóa loss, còn với advantage dương ta muốn tăng log-prob. [Code: RM6, 6.5–6.10]

Nhìn rõ giới hạn ví dụ giảng dạy: `compute_grpo_loss` tại Chương 6 dùng `loss = pg_loss`; tác giả ghi chú *KL term sẽ bổ sung ở chương sau*. Không có *ratio/clipping theo old policy* hay *KL penalty* trong loss đang chạy của notebook này. Sơ đồ **GRPO tổng quát** ở slide 052/062 có thành phần reference/KL. Đây là **hai mức mô tả khác nhau**, không phải bằng chứng một bên viết sai. [Code: RM6, 6.10; Slide 052, tr. 11]

11.3 Ba bài toán vận dụng rất có thể gây nhầm

TH1 – Reward đúng nhưng advantage bằng 0. Với cùng một prompt, cả 4 rollout được reward 1: $\bar{r} = 1$, nên $A_i = 0$ cho tất cả. *Trong loss PG tối giản của notebook*, gradient từ thành phần reward này bằng 0 dù mọi đáp án đều đúng. Điều này không có nghĩa RL không cần đúng; nó chỉ cho thấy *không có tương phản nội nhóm*.

TH2 – Tại sao không `.item()` sớm? Nếu gọi `.item()` trên tổng log-probs *trước khi tính loss*, Tensor thành Python float, đường autograd từ loss về trọng số model bị cắt. Ngược lại, việc dùng `.item()` *để ghi log sau khi đã giữ lại* `loss_tensor` không gây lỗi ấy.

TH3 – Đúng số nhưng sai format. Giả sử đáp án chuẩn là 83. Dạng `\boxed{83}` được 1, nhưng dạng văn bản "The final answer is 83" có thể được 0 theo **hàm reward cụ thể của notebook**, do yêu cầu trích `\boxed{...}`. Đây là mệnh đề về *cách thiết kế reward*, không phải phán xét câu trả lời văn bản sai về toán.

11.4 Bảng tổng kết: hỏi "nếu thay giả thiết thì sao?"

Thay đổi giả thiết	Hệ quả có thể kết luận trong cách triển khai đã đọc
Một nhóm reward giống hệt nhau	Advantages bằng 0; PG loss không mang gradient thưởng-phạt phân biệt các mẫu.
Đổi token trong đáp án mà không đổi prompt	Reward/sequence log-probs có thể đổi; *không* phải đổi tokenizer hoặc kiến trúc.
Dùng `mean` thay `sum` cho sequence logprobs	Thay độ lớn gradient theo độ dài đáp án và không còn là loss được notebook mô tả.
Gọi `no_grad` cả khi tính lại sequence log-probs	Không còn đường gradient cho policy-gradient update.
Thêm KL/ref policy/clipping từ một phiên bản GRPO khác	Thay objective; không được gọi đó là chính xác cùng loss Chương 6.
Không còn bộ xác minh tự động đáng tin cậy	Không thể trực tiếp dùng reward $0/1$ kiểu `reward_rlvr` mà không thiết kế nguồn tín hiệu khác.

12. Checklist 18 ngộ nhận cần xóa trước khi vào thi

1. ID gần nhau $\Rightarrow$ nghĩa gần nhau. **Sai.**
2. Positional embedding luôn nối vào token embedding. **Sai (slide GPT cộng).**
3. Cùng token khác vị trí có input embedding y hệt. **Sai khi có positional info.**
4. Query/Key/Value nhất thiết xuất phát từ ba câu khác nhau. **Sai.**
5. Attention weights phụ thuộc vào V dù Q, K cố định. **Sai.**
6. Softmax attention chạy theo feature dimensions. **Sai trong công thức được dạy.**
7. Causal mask làm token không thấy chính nó.

Sai.

8. Mask sau softmax không cần chuẩn hóa lại. **Sai.**
9. Nhiều head gấp đôi $\Rightarrow$ parameters gấp đôi. **Không tất yếu.**
10. GPT sinh cả câu chỉ bằng argmax một lần. **Sai.**
11. LayerNorm, dropout và residual làm cùng nhiệm vụ. **Sai.**
12. Tăng temperature làm argmax đổi vị trí. **Sai khi** $\tau > 0$**.**
13. Top-k với $k = 1$ vẫn luôn random. **Sai.**
14. Target next-token bằng input cùng vị trí. **Sai.**
15. Classifier 2 lớp vẫn bắt buộc có V logits ở output head. **Sai ở head được thay.**
16. Label -100 nghĩa là input token ID -100. **Sai.**
17. DPO cần train reward model tách riêng giống PPO. **Sai.**
18. GRPO bỏ reward và KL vì bỏ value model. **Sai theo sơ đồ.**

13. Cách xử lý câu tính toán ngắn (không máy tính)

1. **Shape:** ghi các trục trước (B, H, T, S, d_h, V), không đoán bằng cảm giác; nhân ma trận chỉ trên các trục phù hợp.
2. **Parameter:** tách weight và bias, head count và hidden width; $Q/K/V/O$ cơ bản $4d^2$ weight, chưa cộng FFN/embedding.
3. **Probability:** với logits $[a, b]$, nếu $a > b$ thì $p(a) > p(b)$ sau softmax; scale $\tau > 0$ không đổi thứ tự.
4. **Mask:** mask vị trí thành $-\infty$ *trước softmax*; xác suất được phép phải cộng bằng 1. Với label -100, bỏ term loss chứ không bỏ token khỏi forward.
5. **DPO:** so sánh hai tỷ số policy/reference, lấy chênh *log* và dấu. Không cần nhẩm log chính xác để quyết định tốt/xấu tương đối.
6. **Reward group:** điểm nằm trên/dưới trung bình group mới quyết định lợi thế tương đối trong minh họa cơ bản, không lấy mốc 0 tuyệt đối của reward.

14. Đề luyện A – 40 câu trọng tâm, nhiều phương án gần đúng

*Đề tự biên soạn; mỗi câu chọn một đáp án **đúng nhất**. Thời gian luyện gợi ý: 45–55 phút (không phải thời gian thi chính thức). Hãy đánh dấu đáp án mà chưa xem phần giải phía sau.*

Câu 1. Input có $B = 2$, $T = 5$, embedding dimension $d = 8$. Kết quả tra token embedding có shape nào?

A. $[2, 5]$
B. $[2, 5, 8]$
C. $[5, 8, 2]$
D. $[2, 8, V]$

Câu 2. Phát biểu nào đúng nhất về token ID do tokenizer sinh ra?

A. Token có ID gần nhau luôn mang nghĩa gần nhau.
B. ID lớn thì embedding có norm lớn hơn.
C. ID chỉ là chỉ số ánh xạ vào bảng embedding; ý nghĩa phụ thuộc các tham số học được.
D. ID tự chứa vị trí của token trong câu.

Câu 3. Cùng một token xuất hiện tại vị trí 1 và vị trí 4. Theo cách kết hợp embedding trong slide GPT-2, nhận định nào phù hợp?

A. Cùng ID có cùng token embedding, nhưng thêm vị trí nên input có thể khác.
B. Token embedding buộc khác nhau vì vị trí khác.
C. Hai input embeddings phải giống nhau hoàn toàn.
D. Mô hình nối hai vector nên chiều tăng gấp đôi.

Câu 4. Chuỗi $[a, b, c, d, e]$ được dùng huấn luyện next-token với cửa sổ input độ dài 4, bắt đầu từ a. Target phù hợp là gì?

A. $[a, b, c, d]$
B. $[e, e, e, e]$
C. $[d, c, b, a]$
D. $[b, c, d, e]$

Câu 5. Giảm stride của các cửa sổ dữ liệu pretraining thường trực tiếp gây ra điều gì khi các điều kiện khác giữ nguyên?

A. Bắt buộc tăng embedding dimension.
B. Các ví dụ training chồng lắp nhau nhiều hơn.
C. Loại bỏ nhu cầu causal mask.
D. Biến mục tiêu next-token thành masked-language modeling.

Câu 6. Giữ d và T không đổi, tăng vocabulary V. Thành phần nào chắc chắn có chiều liên quan tới V trong mô hình GPT minh họa?

A. Chiều ngang của attention score $T \times T$.
B. Số head attention H.
C. Số hàng token embedding và số logits ở vocabulary output.
D. Số lớp Transformer block.

Câu 7. Trong softmax$(QK^\top/\sqrt{d_k})V$, giữ Q, K cố định nhưng thay V. Điều gì trực tiếp có thể xảy ra?

A. Attention weights giữ nguyên, output context có thể thay đổi.
B. Attention weights buộc thay đổi nhưng context giữ nguyên.
C. Causal mask tự tắt.
D. Kích thước vocabulary phải đổi.

Câu 8. Với self-attention chuẩn, $B = 2$, $T = 3$, $d = 12$, $H = 3$ và chia đều heads, shape attention scores là gì?

A. $[2, 3, 4, 4]$
B. $[2, 12, 3]$
C. $[2, 3, 3, 3]$
D. $[2, 3, 12, 12]$

Câu 9. Lý do cơ bản dùng phép chia $\sqrt{d_k}$ trong scaled dot-product attention là gì?

A. Để attention weights luôn bằng nhau.
B. Để tránh score tăng độ lớn quá mức khi chiều key tăng, giúp softmax ổn định hơn.
C. Để chiều output thành $d_k/2$.
D. Để bỏ softmax.

Câu 10. Đối với query ở vị trí thứ hai (đánh số 1, 2, 3, 4) trong causal self-attention, tập key hợp lệ là:

A. Chỉ vị trí 1.
B. Chỉ vị trí 2.
C. Các vị trí 2, 3, 4.
D. Các vị trí 1 và 2.

Câu 11. Một hàng attention logits sau causal mask có dạng $[0, 0, -\infty]$. Softmax trên hàng này trả ra:

A. $[1/2, 1/2, 0]$
B. $[1/3, 1/3, 1/3]$
C. $[0, 0, 0]$
D. $[1, 1, 0]$

Câu 12. Softmax trong self-attention chuẩn hóa mỗi query chủ yếu theo trục nào?

A. Trục batch.
B. Trục key positions.
C. Trục model layers.
D. Trục vocabulary.

Câu 13. Giữ model width d không đổi và dùng $W_Q, W_K, W_V, W_O \in \mathbb{R}^{d\times d}$. Tăng số head từ 2 lên 4 thì tổng weight matrices trên:

A. Gấp đôi.
B. Giảm một nửa.
C. Giữ nguyên.
D. Bằng số token T.

Câu 14. Cross-attention có $T_{query} = 3$, $S_{key} = 5$, batch 1, $H = 2$. Shape score attention là gì?

A. $[1, 2, 3, 5]$
B. $[1, 2, 5, 5]$
C. $[1, 3, 2, 5, 5]$
D. $[1, 2, 3, 3]$

Câu 15. Trong sơ đồ encoder–decoder additive attention, tại sao context vector c_t có thể khác giữa hai bước sinh t và $t + 1$?

A. Vì encoder phải được pretrain lại sau mỗi bước.
B. Vì số token nguồn luôn thay đổi.
C. Vì vocabulary tự đổi.
D. Vì trạng thái decoder dùng tính alignment thay đổi, nên trọng số trên các hidden states nguồn thay đổi.

Câu 16. Phát biểu nào là đối chiếu phù hợp với hình BERT và GPT trong slide?

A. GPT dùng decoder-only autoregressive; BERT được minh họa theo encoder/masked-word.
B. BERT chỉ có thể dự đoán token kế tiếp.
C. GPT là bidirectional encoder giống BERT.
D. Hai mô hình có masking pattern bắt buộc giống nhau.

Câu 17. Trong block GPT theo sơ đồ slide 041, LayerNorm1 được thực hiện ở đâu so với masked multi-head attention?

A. Sau toàn bộ output head.
B. Trước attention trong nhánh residual (pre-norm).
C. Chỉ trên target labels.
D. Chỉ trước tokenizer.

Câu 18. Vì sao không thể kết luận output cuối của một LayerNorm luôn có mean chính xác bằng 0?

A. Vì LayerNorm là attention.
B. Vì LayerNorm chỉ hoạt động lúc inference.
C. Vì hệ số affine học được (γ, β) có thể thay đổi phân phối sau chuẩn hóa.
D. Vì LayerNorm luôn lấy thống kê từ toàn batch.

Câu 19. Khi chuyển mô hình từ training mode sang evaluation/inference mode chuẩn, phát biểu nào phù hợp nhất về dropout?

A. Tăng tỉ lệ bỏ node để chống overfit.
B. Biến thành causal mask.
C. Bắt buộc đổi vocabulary size.
D. Không còn ngẫu nhiên loại activation theo dropout như lúc train.

Câu 20. Nếu bỏ residual shortcut nhưng giữ kích thước của các phép biến đổi bên trong block, điều nào hợp lý nhất?

A. Gradient có thể truyền khó hơn, dù shape output không nhất thiết thay đổi.
B. Shape output chắc chắn tăng gấp đôi.
C. Causal mask sẽ ngừng hoạt động.
D. Input IDs không còn tồn tại.

Câu 21. GPT nhận batch 2 chuỗi, mỗi chuỗi dài 4 token, vocabulary size 100. Tensor logits toàn chuỗi có shape nào?

A. $[2, 100]$
B. $[2, 4, 100]$
C. $[2, 4, 4]$
D. $[2, 100, 100]$

Câu 22. MHA đơn giản có bốn ma trận Q, K, V, O kích thước 8×8, không tính bias. Tổng số weight là:

A. $8^2 = 64$
B. $3 \times 8 = 24$
C. $4 \times 8^2 = 256$
D. $4 \times 8 = 32$

Câu 23. Với cùng ground-truth token, mô hình P gán xác suất 0.8 và mô hình Q gán 0.2. Đâu là kết luận đúng về negative log-likelihood trên token này?

A. Loss của P thấp hơn Q.
B. Loss của P lớn hơn Q.
C. Bằng nhau vì argmax chưa biết.
D. Không so sánh được nếu không có máy tính.

Câu 24. Trong next-token pretraining, vì sao training thường tính loss ở nhiều vị trí cùng lúc nhưng generation vẫn thêm token lần lượt?

A. Vì trong training attention không cần causal mask.
B. Vì target training hoàn toàn vô dụng.
C. Vì vocab size giảm khi sinh.
D. Vì khi training chuỗi input/target đã có sẵn và được shift, còn generation cần token vừa sinh làm ngữ cảnh tiếp.

Câu 25. Hoạt động nào thuộc bước đánh giá validation trong pipeline chuẩn, thay vì bước cập nhật train?

A. Backward rồi optimizer step trên validation.
B. Đo loss/độ chính xác mà không dùng batch validation để update weights.
C. Xóa gradient của tất cả model trong máy.
D. Buộc sinh thêm token bằng sampling.

Câu 26. Với logits cố định $[3, 2, 1]$, điều gì xảy ra với *argmax* nếu chia cho temperature $\tau = 2 > 0$?

A. Chuyển từ vị trí 1 sang 3.
B. Luôn thành phân phối đều.
C. Vẫn là vị trí 1, mặc dù phân phối softmax phẳng hơn.
D. Bị loại khỏi top-k.

Câu 27. Top-k sampling với $k = 1$ và không có tie về score gần nhất với thao tác nào?

A. Greedy argmax.
B. Chọn đều ngẫu nhiên trên toàn vocabulary.
C. Bỏ hết vocabulary.
D. Chọn token có score thấp nhất.

Câu 28. Để top-k sampling loại hẳn token ngoài tập top-k, cách xử lý điển hình nào đúng?

A. Softmax trên toàn từ vựng rồi giữ nguyên tổng xác suất sau khi zero ngoài top-k.
B. Tăng dropout lên 100%.
C. Mã hóa tất cả token về cùng ID.
D. Gán logits ngoài top-k bằng $-\infty$ trước softmax, rồi lấy mẫu.

Câu 29. Trong ví dụ fine-tune GPT phân loại spam/not spam, thay đổi head có ý nghĩa gì?

A. Head vẫn dự đoán token trên $V = 50{,}257$ lựa chọn.
B. Head cuối ánh xạ hidden dimension về 2 class logits thay vì vocabulary logits.
C. Mọi transformer block bị thay bằng tokenizer.
D. Causal attention buộc thay thành cross-attention.

Câu 30. Với sơ đồ classification-finetuned GPT ở slide, nhận định nào chính xác hơn?

A. Phải hỏi model bằng chuỗi "Trả lời spam hay không mỗi lần.
B. Không thể tận dụng pretrained weights.
C. Model có thể nhận text trực tiếp và xuất nhãn class theo head được fine-tune.
D. Model bắt buộc sinh toàn bộ câu bằng autoregression.

Câu 31. Chuyển một mẫu instruction dataset từ template Alpaca sang Phi-3 (giữ nội dung đúng) có tác động chắc chắn nào?

A. Đổi hình thức tuần tự hóa/đánh dấu vai trò, không tự thay đổi thông tin supervision gốc.
B. Buộc đổi nhãn phân loại từ 2 lên 3.
C. Đảm bảo chất lượng cao hơn mọi template.
D. Biến mô hình thành reward model.

Câu 32. Trong instruction fine-tuning, gán -100 cho labels của instruction nghĩa là:

A. Xóa instruction khỏi input forward.
B. Biến instruction thành token mới ID -100.
C. Cấm model đọc instruction qua attention.
D. Bỏ qua loss tại các vị trí instruction được chỉ định.

Câu 33. Vì sao việc mask loss trên instruction nhưng giữ loss trên response có thể hợp lý?

A. Để model không nhận biết instruction.
B. Để ưu tiên tối ưu khả năng sinh phần câu trả lời trong khi vẫn conditioning trên prompt.
C. Để tăng context window vô hạn.
D. Để biến cross entropy thành PPO.

Câu 34. Hai câu trả lời đều đúng về mặt kỹ thuật nhưng một câu dễ hiểu và phù hợp người dùng hơn. Giai đoạn nào tập trung xử lý sở thích tương đối kiểu này?

A. Chỉ tokenizer training.
B. Chỉ thay position embedding.
C. Preference alignment (RLHF/DPO).
D. Chỉ thêm hidden layers không cần tín hiệu ưu tiên.

Câu 35. Thứ tự ba bước trong sơ đồ RLHF truyền thống là gì?

A. SFT $\rightarrow$ thu preference/train reward model $\rightarrow$ PPO cập nhật policy.
B. PPO $\rightarrow$ tokenizer $\rightarrow$ SFT.
C. Chỉ DPO, không dùng dữ liệu preference.
D. Train classifier $\rightarrow$ bỏ pretrained model $\rightarrow$ inference.

Câu 36. Reward model được học từ human preference/ranking chủ yếu làm gì?

A. Đảm bảo mọi câu có tính đúng đắn tuyệt đối.
B. Xóa toàn bộ bias đánh giá của con người.
C. Thêm chiều token vào vocabulary.
D. Ước lượng mức được ưa thích/hợp mục tiêu của câu trả lời để làm tín hiệu tối ưu.

Câu 37. Khác biệt cốt lõi của DPO so với pipeline RLHF PPO trong sơ đồ slide là:

A. DPO không cần cặp preference.
B. DPO tối ưu policy trực tiếp từ preference pairs mà không cần train một explicit reward model và chạy PPO.
C. DPO chỉ dùng masked-language modeling.
D. DPO không dùng reference policy.

Câu 38. Trong DPO, π_{ref} trong công thức đóng vai trò gì?

A. Mô hình phân loại 2 nhãn.
B. Danh sách token bị mask.
C. Phân phối tham chiếu dùng so sánh tỷ số xác suất chosen/rejected.
D. Số lượng heads.

Câu 39. Trên slide so sánh Llama 3.2 1B và Qwen3 0.6B, mệnh đề nào đúng?

A. Qwen sâu hơn (28 block so với 16), Llama rộng hơn (embedding 2048 so với 1024).
B. Qwen nông hơn và rộng hơn.
C. Cả hai có số blocks và embedding dimension giống nhau.
D. Số parameters lớn hơn luôn kéo theo số layers lớn hơn.

Câu 40. Khác nhau về biểu diễn vị trí giữa GPT-2 minh họa và các Llama sử dụng RoPE ở slide là:

A. RoPE làm model không còn cần thông tin vị trí.
B. RoPE bắt buộc nối ID với vocabulary.
C. GPT-2 dùng cùng một vị trí cho mọi token.
D. GPT-2 minh họa cộng positional embedding tuyệt đối vào token embedding; Llama dùng RoPE trong attention.

15. Đề luyện B – 40 câu vận dụng và phản thực tế

Đề tự biên soạn, độ khó nhỉnh hơn đề A. Một số câu yêu cầu hai bước lập luận (thay một giả thiết rồi dự đoán hệ quả). Không cần máy tính.

Câu 1. Một tokenizer hoán đổi ID của hai token, đồng thời người triển khai hoán đổi đúng hai hàng tương ứng trong bảng embedding, các phần khác giữ nguyên. Kết quả tra embedding đối với văn bản cũ sẽ thế nào?

A. Luôn đổi vì ID mang sẵn nghĩa ngữ pháp.
B. Có thể giữ nguyên nếu phép hoán vị bảng và ID nhất quán.
C. Chiều embedding bắt buộc tăng.
D. Phải tăng V thêm 2.

Câu 2. Giả sử token IDs tại vị trí 1 và 3 đều bằng 7; học positional embedding là $p_1 \neq p_3$. Điều nào suy ra trực tiếp từ input embedding kiểu GPT-2?

A. Input embeddings bằng nhau vì cùng ID.
B. Positional embeddings không tham gia forward.
C. Chắc chắn cosine similarity hai vector bằng 0.
D. Input embeddings $e_7 + p_1$ và $e_7 + p_3$ khác nhau.

Câu 3. Chọn nhận định phân biệt đúng ba phép softmax: attention, language modeling, và classification.

A. Theo thứ tự, chúng phân phối trên keys, token vocabulary và classes.
B. Cả ba đều phân phối trên batch.
C. Attention phân phối trên vocabulary còn LM trên key positions.
D. Classification hai lớp phải có V giá trị trước softmax.

Câu 4. Trong training next-token của decoder-only GPT, target ở vị trí i là token $i+1$ trong văn bản. Điều nào giúp tránh sử dụng trái phép thông tin tương lai?

A. Xóa hết target trong training.
B. Chỉ dùng hai token trong vocabulary.
C. Causal mask ngăn hidden state vị trí i truy cập input tương lai, dù loss ở nhiều vị trí được tính đồng thời.
D. Đặt tất cả logits bằng nhau.

Câu 5. Hai mô hình có cùng Q, K và cùng causal mask. Mô hình 1 đổi V của key thứ ba nhưng key thứ ba bị mask cho một query cụ thể. Output của query ấy có thay đổi vì thao tác đó không?

A. Có, vì mọi value đều cộng vào context vector.
B. Không, do trọng số ở key bị mask là 0.
C. Có, vì mask chỉ áp dụng sau khi output.
D. Bắt buộc không xác định vì d_k chưa biết.

Câu 6. Một người tính softmax score $[1, 1, 1]$ ra $[1/3, 1/3, 1/3]$, rồi đặt vị trí 3 bằng 0 để giả lập causal mask. Đâu là lỗi?

A. Ba scores phải được nhân V trước.
B. Causal mask không được phép dùng.
C. Cách đó luôn đúng dù tổng xác suất giảm.
D. Cần mask trước softmax (hoặc chuẩn hóa lại) để được $[1/2, 1/2, 0]$.

Câu 7. Trong causal self-attention độ dài $T = 5$, query ở vị trí số 3 (đếm từ 1) được xem tối đa mấy vị trí key nếu không có padding?

A. 3 vị trí: 1, 2 và 3.
B. 2 vị trí: 1, 2.
C. 5 vị trí.
D. Chỉ vị trí 3.

Câu 8. Encoder nguồn dài 5, decoder query dài 2, batch 1, $H = 4$. Attention score của cross-attention có shape?

A. $[1, 4, 5, 5]$
B. $[1, 4, 2, 2]$
C. $[1, 4, 2, 5]$
D. $[1, 2, 4, 5, 5]$

Câu 9. Model width $d = 12$ chia đều thành $H = 3$ heads. Trong scaled dot-product attention mỗi head, mẫu số scaling là bao nhiêu?

A. $\sqrt{12}$
B. $\sqrt{4} = 2$
C. 3
D. 12

Câu 10. Hai heads tự chú ý nhận cùng input X nhưng khác W_Q, W_K, W_V. Có nhất thiết tạo attention weights giống nhau không?

A. Có, vì input giống nhau.
B. Có, vì mỗi head đều có softmax.
C. Có, nếu T giống nhau.
D. Không, vì phép chiếu học được riêng có thể dẫn tới score và weights khác nhau.

Câu 11. Nếu mọi value vectors V_i trong một hàng attention đều bằng một vector v, output weighted-sum của hàng này (weights không âm, tổng 1) là gì?

A. Bằng v bất kể phân bố attention weights.
B. Bằng $T \cdot v$.
C. Bằng 0 do các weights triệt tiêu.
D. Bằng một vector tùy ý không suy ra được.

Câu 12. Giữ B, H, d_h cố định, tăng độ dài chuỗi tự chú ý từ T lên $2T$. Số phần tử trong tensor attention scores $[B, H, T, T]$ thay đổi ra sao?

A. Gấp 2.
B. Không đổi.
C. Gấp 4.
D. Gấp 8.

Câu 13. Trong GPT block chuẩn ở slide, module nào chủ yếu trộn thông tin giữa các vị trí token khác nhau (thay vì chỉ biến đổi features theo từng vị trí)?

A. LayerNorm đơn lẻ.
B. Masked self-attention.
C. GELU đơn lẻ.
D. Output vocabulary softmax.

Câu 14. Slide mô tả GELU trơn hơn ReLU tại vùng quanh 0. Kết luận nào hợp lý?

A. GELU chắc chắn tối ưu hơn trong mọi ứng dụng.
B. ReLU không thể train với gradient descent.
C. GELU khiến dropout vô nghĩa.
D. Độ trơn có thể thuận lợi cho tối ưu trong một số trường hợp, nhưng không bảo đảm vượt ReLU mọi lúc.

Câu 15. Một người nói: "LayerNorm bắt buộc cần thống kê toàn batch nên inference batch 1 không thể chạy. Sai ở đâu?

A. LayerNorm trong kiến trúc minh họa dùng thống kê theo feature của từng mẫu/token, không phải cần thống kê toàn batch.
B. LayerNorm chính là top-k.
C. Vì LayerNorm chỉ dùng khi train reward model.
D. Vì LayerNorm không có tham số affine.

Câu 16. Hai lần sinh từ cùng prompt ở inference mode cho kết quả khác nhau. Kết luận nào *không* nên rút ra ngay?

A. Decoding có thể đang dùng sampling.
B. Có thể temperature/top-k làm đầu ra có tính ngẫu nhiên.
C. Khác biệt chắc chắn do dropout vẫn hoạt động dù inference mode.
D. Cần kiểm tra cấu hình decoding.

Câu 17. Giữ model width d và context T không đổi, tăng V từ 100 lên 200. Thành phần nào không buộc thay shape?

A. Output logits LM theo vocabulary.
B. Token embedding table.
C. LM output projection sang vocabulary.
D. Attention score $[B, H, T, T]$.

Câu 18. MHA đầy đủ có 4 projection $d \times d$. Nếu d tăng từ 8 lên 16 (bias bỏ qua), số weight của các projection tăng mấy lần?

A. 2 lần.
B. 4 lần.
C. 8 lần.
D. Không tăng.

Câu 19. Logits next-token là $[4, 2, 0]$. Nếu dùng greedy và chỉ thay temperature từ 1 sang 0.5, token nào được chọn?

A. Token thứ nhất cả hai lần.
B. Token thứ hai khi $\tau = 0.5$.
C. Token thứ ba khi $\tau = 0.5$.
D. Không dự đoán được dù không tie.

Câu 20. Khi đổi từ greedy decoding sang temperature sampling $\tau = 1$ mà không giới hạn top-k, điều nào có thể xảy ra?

A. Output luôn giống greedy vì $\tau = 1$.
B. Vocabulary bị giảm còn một token.
C. Token không phải top-1 vẫn có thể được chọn nếu xác suất khác 0.
D. Mọi tokens có xác suất bằng nhau.

Câu 21. Một vector logits có các giá trị phân biệt $[7, 1, 5, 3]$, dùng top-k với $k = 2$. Những vị trí nào còn cơ hội được chọn?

A. 1 và 2.
B. 2 và 4.
C. 3 và 4.
D. 1 và 3.

Câu 22. Hai phân phối khác nhau gán cùng xác suất 0.5 cho token đúng của một vị trí. Với cross entropy one-hot trên đúng vị trí đó, loss của hai phân phối có so sánh thế nào?

A. Khác nhau tùy xác suất các token sai.
B. Bằng nhau: cùng $-\log(0.5)$.
C. Mô hình có vocabulary lớn hơn luôn loss cao hơn.
D. Không thể tính loss vì chưa biết head count.

Câu 23. Tập train có 25 mẫu, batch size 5, mỗi epoch đi hết tập một lượt và không dùng gradient accumulation. Số lần optimizer update trên một epoch là bao nhiêu?

A. 25.
B. 1.
C. 5.
D. 125.

Câu 24. Giả sử train loss giảm liên tục nhưng validation loss tăng mạnh về sau. Cách diễn giải phù hợp nhất?

A. Có dấu hiệu mô hình đang overfit, cần xem khả năng generalization.
B. Chắc chắn code tính loss sai.
C. Chắc chắn model chưa học gì.
D. Nên tối ưu trực tiếp trên validation batches bằng gradient để “chữa hiện tượng.

Câu 25. Cùng một instruction "dịch sang tiếng Đức và output đúng được bọc bằng hai chat templates khác nhau. Kết luận nào chính xác?

A. Nhất thiết hai tasks khác nhau.
B. Template nào dài hơn thì labels chuẩn xác hơn.
C. Chuyển template không cần quan tâm tokenizer.
D. Vẫn cùng supervision về nội dung, nhưng cách tokenize/role delimiters của đầu vào thay đổi.

Câu 26. Khi instruction tokens không được tính loss (labels -100) nhưng vẫn có trong input, mô hình có thể dùng instruction để sinh response không?

A. Không, vì -100 chặn cả self-attention.
B. Có, vì loss mask không xóa chúng khỏi context forward.
C. Không, trừ khi dùng PPO.
D. Có, nhưng chỉ nếu $V = 100$.

Câu 27. Một batch được padding đến cùng chiều dài. Để pad không góp target loss, slide gợi ý thao tác nào?

A. Thay nhãn tương ứng vị trí pad bằng -100 (với ignore-index tương ứng).
B. Giữ nhãn pad như class thứ ba.
C. Xóa softmax attention khỏi model.
D. Tăng d_k theo số pad.

Câu 28. Một classifier đã được fine-tune theo sơ đồ, có head 2 lớp. Người dùng lấy logits $[B, 2]$ để làm top-k trên từ vựng hàng chục nghìn token. Nhận xét đúng?

A. Hoàn toàn tương đương next-token LM.
B. Head phải tự biến thành $[B, V]$ khi có top-k.
C. Đang nhầm trục class của classification với trục vocabulary của LM.
D. Top-k bắt buộc là bước train classifier.

Câu 29. Khi đổi GPT pretrained thành model classification, phát biểu nào đúng nhất về vai trò của pretrained backbone?

A. Pretrained weights buộc bỏ hết.
B. Không bao giờ được cập nhật backbone khi fine-tune.
C. Bắt buộc giữ nguyên cả LM head vocabulary.
D. Cấu trúc biểu diễn đã học được tái sử dụng, thay head và điều chỉnh tham số theo cấu hình fine-tune.

Câu 30. Trong một preference pair, $R_w = \pi_\theta(y_w|x)/\pi_{\text{ref}}(y_w|x) = 1/2$ và $R_l = 2$. Khi $\beta > 0$, margin DPO là:

A. Dương vì response được chọn luôn có ưu tiên.
B. Bằng 0.
C. Âm vì $\log(1/2) - \log 2 < 0$.
D. Không xác định nếu thiếu optimizer.

Câu 31. Tăng β trong cùng công thức DPO khi các log-ratios cố định gây ra điều gì trực tiếp?

A. Co giãn độ lớn của preference margin trước sigmoid; không đơn thuần đổi learning rate.
B. Xóa reference policy.
C. Bắt buộc chia vocabulary size hai lần.
D. Đảo nhãn chosen và rejected.

Câu 32. Trong hai policy khác nhau, nếu với cùng cặp (x, y_w, y_l) cả chosen và rejected đều có tỷ số policy/reference ***bằng nhau*** $R_w = R_l > 0$, preference margin DPO cho pair này là gì?

A. Luôn dương.
B. Bằng 0.
C. Luôn âm.
D. Bằng βV.

Câu 33. Phát biểu nào diễn tả đúng vai trò của reference model trong DPO với công thức trên slide?

A. Chỉ dùng để tạo nhãn spam.
B. Tự động được cập nhật gradient cùng policy trong mọi thiết lập.
C. Được loại bỏ hoàn toàn vì DPO "direct.
D. Cho các xác suất nền để so sánh mức tăng/giảm xác suất chosen/rejected của policy đang học.

Câu 34. Trong sơ đồ PPO của bài RL, value model có chức năng quan trọng nào?

A. Ước lượng baseline/value cho việc tính advantage (GAE trong sơ đồ).
B. Đóng vai trò tokenizer.
C. Thay thế hoàn toàn policy model.
D. Đếm vocabulary tokens.

Câu 35. Sơ đồ GRPO lấy nhiều output từ cùng một prompt để làm gì?

A. Bắt buộc tăng V.
B. Thay thế causal mask bằng full attention.
C. Sử dụng rewards của một nhóm để tính group-relative advantages cho tối ưu policy.
D. Chỉ lấy average logits trước tokenizer.

Câu 36. Giả sử 3 output cùng prompt có rewards bằng nhau. Theo ý tưởng relative advantage trong nhóm, điều nào hợp lý?

A. Một response chắc chắn có advantage lớn nhất.
B. Không có response nào nổi bật theo so sánh *trong nhóm* chỉ dựa vào ba rewards này.
C. Reward model phải lập tức thành classifier.
D. PPO và DPO trở thành cùng công thức.

Câu 37. Nếu dữ liệu ranking ưu tiên lời giải dài và dễ đọc hơn dù có lỗi factual nhỏ, reward model học từ ranking đó chắc chắn làm được gì?

A. Luôn khôi phục sự thật khách quan.
B. Không bao giờ bị sai lệch bởi preferences.
C. Đảm bảo giảm hallucination tới 0.
D. Có thể phản ánh thiên lệch preference của dữ liệu; không thể xem là bộ kiểm chứng đúng/sai tuyệt đối.

Câu 38. Sơ đồ Llama 3.2 1B nhắc tới “weight tying. Hệ quả nào hợp lý nhất?

A. Embedding và output projection có thể dùng chung trọng số tương ứng; output vẫn có vocabulary logits.
B. Bỏ hoàn toàn output head nên không sinh next token.
C. Mỗi head Q/K/V trở thành cùng một head.
D. Buộc bỏ tất cả positional information.

Câu 39. Điểm nào được minh họa *riêng trên sơ đồ Qwen3 0.6B* so với Llama 3.2 1B ở trang so sánh?

A. Qwen hoàn toàn không có RoPE.
B. Qwen không có attention.
C. Cụm Q/K RMSNorm bổ sung trong nhánh attention.
D. Qwen không dùng SiLU/gating.

Câu 40. Chọn kết luận thận trọng nhất khi một phiên bản LLM có context window dài hơn phiên bản khác.

A. Số lượng weight matrices Q/K/V/O chắc chắn tăng tuyến tính theo max context.
B. Model chắc chắn chính xác hơn trên mọi tác vụ.
C. Số vocab tokens chắc chắn tăng gấp đôi.
D. Model có thể hỗ trợ nhiều vị trí ngữ cảnh hơn, nhưng không suy ra ngay số tham số hay chất lượng tổng quát.

16. Đề luyện C – 40 câu bám sát code Raschka và notebook RL

*Đề tự biên soạn từ mã nguồn thực tế, nặng về đọc lệnh, phản ví dụ, shape, nhầm lẫn giữa phiên bản giản lược và công thức tổng quát. Chọn **một đáp án đúng nhất**. Các ký hiệu R2–RE, RM6 chỉ nguồn ở "Bản đồ nguồn GitHub phía trước.*

Câu 1. `GPTDatasetV1` nhận 12 token, `max_length=4`, `stride=3`. Với đúng vòng lặp trong `gpt.py`, số cửa sổ được tạo là?

A. 2
B. 3
C. 4
D. 8

Câu 2. Trong một cửa sổ next-token, chuỗi gốc có bốn ID $[5, 6, 7, 8]$, input là $[5, 6, 7]$. Target đúng là gì (không phụ thuộc stride giữa các mẫu)?

A. $[5, 6, 7]$
B. $[5, 7, 8]$
C. $[8, 8, 8]$
D. $[6, 7, 8]$

Câu 3. Một GPT minh họa có $V = 100, d = 8, L = 16$. Chỉ tính hai bảng *token embedding* và *learned positional embedding*, bao nhiêu tham số weight?

A. 116
B. 800
C. 928
D. 1280

Câu 4. Cho token IDs shape $[2, 4]$ và embedding width $d = 6$. Sau lookup token embedding, shape là?

A. $[2, 4, 6]$
B. $[2, 4, V]$
C. $[2, 6]$
D. $[4, 6, 2]$

Câu 5. Hai vị trí trong cùng câu có cùng token ID, nhưng learned positional embeddings khác nhau. Trong `GPTModel`, điều nào đúng?

A. Input vectors luôn trùng nhau vì ID giống nhau.
B. Model bỏ qua vị trí khi ID lặp lại.
C. Token lookup giống nhau nhưng tổng token+position có thể khác nhau.
D. Vị trí làm thay đổi ID trước tokenizer.

Câu 6. Ở causal self-attention với $T = 5$, query của token tại chỉ số $i = 2$ (đánh số từ 0) được phép nhìn bao nhiêu vị trí?

A. 2
B. 3
C. 4
D. 5

Câu 7. Trong mã `MultiHeadAttention`, input có $B = 2, T = 6, d_{out} = 12, H = 3$. Shape của Q sau chia head và shape scores lần lượt là?

A. $[2, 6, 3, 4]$ và $[2, 6, 6]$
B. $[2, 3, 6, 12]$ và $[2, 3, 12, 12]$
C. $[2, 3, 4, 6]$ và $[2, 6, 4, 4]$
D. $[2, 3, 6, 4]$ và $[2, 3, 6, 6]$

Câu 8. Khởi tạo `MultiHeadAttention(d_out=10, num_heads=3)` sẽ gặp vấn đề gì ngay theo mã nguồn?

A. Vấp `assert` vì 10 không chia hết cho 3.
B. Chạy được, tự động làm tròn d_h lên 4.
C. Tăng số heads lên 10.
D. Bắt buộc đổi vocabulary size.

Câu 9. Giữ $d_{in} = d_{out} = 12$ và các phép chiếu tuyến tính Q/K/V/O như code, chỉ đổi heads $3 \to 4$. Nhận định nào chính xác nhất?

A. Số weight Q/K/V/O tăng tỷ lệ $4/3$.
B. Tổng weight projection không đổi; d_h giảm từ 4 còn 3.
C. Output đổi chiều từ 12 lên 16.
D. Bốn head buộc chia sẻ một ma trận Q.

Câu 10. Trong `MultiHeadAttention` của Raschka, vì sao phát biểu "mỗi hàng attention weights luôn cộng lại đúng 1 chưa chính xác?

A. Vì softmax chuẩn hóa theo batch.
B. Vì softmax được dùng cho values thay vì scores.
C. Vì dropout áp sau softmax trong training có thể đổi tổng hàng.
D. Vì causal mask làm mọi hàng có tổng 0.

Câu 11. Trong cấu hình `qkv_bias=False`, điều nào vẫn đúng?

A. Q/K/V là đầu ra hằng, không có tham số.
B. Output projection cũng bắt buộc không bias.
C. Causal mask bị tắt.
D. Ba ma trận trọng số Q/K/V vẫn được học; chỉ bias Q/K/V bị bỏ.

Câu 12. Nếu giữ Q, K cố định và chỉ thay V trong một forward attention, nhận định nào đúng?

A. Attention scores và softmax weights không đổi, nhưng context có thể đổi.
B. Attention scores bắt buộc thay đổi.
C. Mask không còn hiệu lực.
D. Đầu ra bắt buộc không đổi.

Câu 13. Công thức nào phản ánh *thứ tự pre-norm và residual* trong attention sub-block của `TransformerBlock`?

A. $\text{LN}(x + \text{Attn}(x))$
B. $\text{Attn}(x) + \text{LN}(\text{Attn}(x))$
C. $x + \text{Drop}(\text{Attn}(\text{LN}(x)))$
D. $\text{Drop}(x) + \text{LN}(\text{Attn}(x))$

Câu 14. `LayerNorm` trong code áp trên $[B, T, d]$ và dùng `unbiased=False`. Mệnh đề nào đúng?

A. Nó chuẩn hóa duy nhất theo trục batch B.
B. Nó lấy mean/variance theo chiều cuối d của từng token và dùng phương sai không hiệu chỉnh.
C. Nó loại bỏ nhu cầu scale và shift học được.
D. Nó bắt buộc làm toàn tensor có tổng đúng 0.

Câu 15. Một người nói: `model.eval()` tương đương hoàn toàn với `torch.no_grad()`. Phản biện đúng nhất là?

A. Cả hai đều tắt dropout và autograd.
B. `no_grad()` tự chuyển toàn bộ model sang eval.
C. `eval()` xóa gradient đang lưu.
D. `eval()` đổi hành vi layer như dropout; `no_grad()` tắt ghi đồ thị gradient.

Câu 16. Vì sao `generate_text_simple` chỉ lấy logits `[:, -1, :]` nhưng lúc pretraining không nhất thiết chỉ học một vị trí?

A. Generation cần dự đoán token kế tiếp từ vị trí cuối; training tính các next-token targets đã dịch trên nhiều vị trí.
B. Pretraining chỉ cập nhật lớp embedding.
C. Logits của mọi vị trí trước đều bằng nhau.
D. Vì vocab luôn có hai token.

Câu 17. `generate` nhận prompt 6 tokens, `context_size=4`, sinh thêm 3 tokens. Điều nào phù hợp với hàm mã nguồn?

A. Hàm báo lỗi ngay vì prompt dài hơn 4.
B. Mô hình bắt buộc trả tổng cộng 4 tokens.
C. Mỗi forward lấy tối đa 4 tokens gần nhất, còn tensor kết quả vẫn có thể dài 9 tokens.
D. Mỗi forward nhìn đủ 9 tokens cả khi context size chỉ 4.

Câu 18. Cho logits phân biệt rõ và temperature $\tau > 0$. Khi tăng τ từ 0.5 lên 2 trước softmax, nhận định nào đúng nhất?

A. Thứ hạng logits tự đảo ngược.
B. Phân phối bớt sắc, nhưng thứ hạng logits không đổi.
C. Vocab giảm kích thước.
D. Mô hình chuyển từ lấy mẫu sang beam search.

Câu 19. Với top-k ($k = 3$) và temperature dương, phát biểu nào diễn tả đúng code?

A. Bắt buộc chọn top-1.
B. Tất cả vocabulary vẫn có xác suất dương.
C. Ba token được chia đều xác suất bất kể logits.
D. Chỉ giữ các token qua ngưỡng top-3 để sampling; xác suất giữa chúng vẫn phụ thuộc logits.

Câu 20. Giả sử logit lớn nhất là *duy nhất*. Đặt `top_k=1` và `temperature=2` trong `generate` thì kết quả là?

A. Vẫn chọn token có logit lớn nhất với xác suất 1 trong tập được phép.
B. Tự chọn token nhỏ nhất vì nhiệt độ lớn.
C. Bắt buộc sinh EOS.
D. Không thể chạy softmax sau top-k.

Câu 21. Sau khi thay GPT LM head bằng classifier $d \to 2$, với $B = 4, T = 10$, logits tính *classification loss* trong code có shape nào?

A. $[4, 10, V]$
B. $[4, 10, 2]$ (trực tiếp đưa vào loss)
C. $[4, 2]$
D. $[10, 2]$

Câu 22. `SpamDataset` pad các chuỗi ngắn đến cùng độ dài; classifier dùng `model(x)[:, -1, :]`. Điều nào đúng nhất?

A. Hàm tự xác định vị trí non-PAD cuối trước khi slicing.
B. Vì PAD bị xóa khỏi input ngay từ dataset nên không ảnh hưởng.
C. `-1` nghĩa là class 1.
D. `-1` là vị trí tensor cuối, có thể tương ứng PAD với chuỗi ngắn.

Câu 23. Trong script Chương 6, sau khi đóng băng model và thay output head, những phần backbone nào được mở khóa trở lại?

A. Transformer block cuối và `final_norm`.
B. Mọi transformer block.
C. Chỉ token embedding.
D. Không có thành phần nào ngoài output head.

Câu 24. Bộ training spam trong code được cân bằng nhãn bằng cách lấy mẫu bớt ham. Có thể suy ra điều nào về bài toán triển khai thực tế?

A. Phân bố spam ngoài đời chắc chắn 50%.
B. Accuracy trên data cân bằng không tự cho biết performance trên phân bố thực tế lệch nhãn.
C. Model không còn cần validation.
D. Cross entropy chỉ hoạt động với data cân bằng.

Câu 25. Trong `custom_collate_fn`, giả sử một target đã padding có ba vị trí mang `pad_token_id`. Theo quy tắc mask trong script, bao nhiêu vị trí trong ba vị trí đó được chuyển thành `-100`?

A. 0
B. 1
C. 2
D. 3

Câu 26. Khi một vị trí target có giá trị `-100` với `ignore_index=-100`, điều gì được bảo đảm?

A. Attention tại vị trí đó bị đặt bằng 0.
B. Token ID `-100` sẽ đi qua embedding table.
C. Model không thực hiện forward tại vị trí đó.
D. Vị trí đó không góp vào cross-entropy loss; attention là vấn đề khác.

Câu 27. Trong pipeline instruction fine-tuning cơ bản của script, dữ liệu là instruction + response. Phát biểu nào đúng?

A. Mặc định collate tạo target bằng phép dịch một token; không tự loại loss mọi token prompt.
B. Chỉ response được tokenized, prompt biến mất.
C. Câu lệnh instruction không được đưa vào model.
D. `ignore_index` mặc định che toàn bộ instruction.

Câu 28. Áp dụng LoRA cho linear có $d_{in} = 12, d_{out} = 8$, rank $r = 2$ và không train bias mới. Số tham số adapter của một linear là?

A. 20
B. 40
C. 96
D. 192

Câu 29. Vì sao ngay sau khởi tạo `LoRALayer` của Appendix E, kết quả nhánh LoRA bằng 0 dù A không bằng 0?

A. $\alpha = 0$ trong mọi trường hợp.
B. Base weight đã bị xóa.
C. B được khởi tạo toàn số 0, nên $xAB = 0$.
D. `rank` bắt buộc 0.

Câu 30. Trong công thức LoRA của Appendix E, nếu $\alpha = 8$, $r = 4$, hệ số nhân trước nhánh xAB là?

A. $1/2$
B. 1
C. 4
D. 2

Câu 31. Giữ d_{in}, d_{out} cố định, tăng rank LoRA từ $r = 2$ thành $r = 4$. Điều nào suy ra trực tiếp?

A. Số tham số adapter tăng gấp đôi, nhưng chất lượng và output không chắc tăng gấp đôi.
B. Trainable parameters giảm một nửa.
C. Số layers Transformer bắt buộc gấp đôi.
D. Tokenizer vocabulary phải tăng.

Câu 32. Appendix E của repo `LLMs-from-scratch` trực tiếp triển khai cơ chế nào?

A. Bắt buộc lượng tử hóa base model 4-bit rồi train toàn bộ weight.
B. Freeze base và bổ sung các ma trận low-rank LoRA; không phải bản triển khai đầy đủ của QLoRA.
C. Chỉ thay tokenizer bằng BPE mới.
D. RLHF với reward model.

Câu 33. Notebook RL Chương 6 phân biệt training-time và inference-time scaling. Chọn phát biểu chính xác nhất.

A. Hai khái niệm luôn đồng nghĩa.
B. Inference-time scaling bắt buộc cập nhật weights.
C. Training-time scaling tăng compute lúc huấn luyện, inference-time scaling dùng thêm compute lúc tạo đáp án; có thể kết hợp.
D. RL chỉ là một loại prompt engineering.

Câu 34. Khác biệt chủ yếu giữa RLHF kiểu học reward từ human preferences và RLVR trong notebook là gì?

A. RLVR phải có nhãn chọn–bỏ do người đánh giá cho mỗi cặp.
B. RLVR chỉ được dùng cho dữ liệu không có đáp án.
C. RLHF không thể dùng reward model.
D. RLVR thay reward model học từ người bằng tín hiệu kiểm chứng tự động đối với bài toán phù hợp.

Câu 35. Giả sử ground truth là 83. Hàm `reward_rlvr` trong notebook có yêu cầu `boxed{...}` (không fallback). Cặp nào có reward đúng theo ví dụ?

A. `boxed{83}` được 1, câu *The final answer is 83* không boxed được 0.
B. Cả hai được 1 vì số đều đúng.
C. Cả hai được 0 vì không có reward model học được.
D. `boxed{38}` được 1 vì đúng cú pháp.

Câu 36. Notebook dùng `no_grad` khi sample rollout, nhưng gọi lại model để tính `sequence_log prob` với autograd. Vì sao?

A. Vì mỗi rollout đã trực tiếp chứa gradient khả vi qua sampling.
B. Để sinh tiết kiệm bộ nhớ, nhưng policy-gradient vẫn cần gradient của log-prob của các token đã sinh.
C. Để backprop qua `grade_answer` vốn không khả vi.
D. Vì `torch.no_grad` tự đóng băng weights vĩnh viễn.

Câu 37. Một prompt cho 4 rollout với rewards $[1, 1, 0, 0]$. Với advantage chuẩn hóa nội nhóm, dấu của advantages theo thứ tự là?

A. $[+, +, +, +]$
B. $[0, 0, 0, 0]$
C. $[+, +, -, -]$
D. $[-, -, +, +]$

Câu 38. Nếu tất cả rewards của cùng một group là $[1, 1, 1, 1]$, điều gì xảy ra với *tín hiệu reward-gradient của loss tối giản* trong Chương 6?

A. Advantage đều dương vì reward=1.
B. Loss bắt buộc đẩy xác suất mọi rollout về 0.
C. Softmax tự chuyển sang temperature=0.
D. Ưu thế từng mẫu bằng 0, nên policy-gradient từ advantage không tạo lực cập nhật phân biệt.

Câu 39. Khi tính `sequence_logprob` để đưa vào loss của notebook, vì sao thay `sum` bằng `mean` không tương đương?

A. Vì $\sum_t \log p(y_t \mid \cdot)$ là log-xác suất cả chuỗi, còn mean chia thêm cho chiều dài và thay độ lớn tín hiệu gradient.
B. Vì `sum` làm mọi xác suất bằng 1.
C. Vì `mean` tự tắt causal mask.
D. Vì `sum` chỉ hợp lệ khi $T = 1$.

Câu 40. Phát biểu nào *khớp với loss hiện được thực thi trong notebook RL Chương 6*, và đồng thời không nhầm với GRPO tổng quát ở sơ đồ slide?

A. Notebook triển khai full PPO với value head và GAE.
B. Notebook dùng $-\text{mean}(A_i \log \pi_\theta(y_i \mid x))$; chưa thêm KL và không dùng ratio/clipping old-policy trong loss này.
C. Notebook chỉ làm DPO trên pairs chosen/rejected.
D. Notebook backprop trực tiếp qua hàm chấm đáp án rời rạc.

17. Đáp án và lời giải – Đề A

Mỗi lời giải chỉ rõ cơ chế và trang slide có liên quan; trả lời đúng bằng may mắn nhưng sai về lập luận vẫn nên xem là chưa nắm chắc.

1:B	**2:C**	**3:A**	**4:D**	**5:B**	**6:C**	**7:A**	**8:C**	**9:B**	**10:D**
11:A	**12:B**	**13:C**	**14:A**	**15:D**	**16:A**	**17:B**	**18:C**	**19:D**	**20:A**
21:B	**22:C**	**23:A**	**24:D**	**25:B**	**26:C**	**27:A**	**28:D**	**29:B**	**30:C**
31:A	**32:D**	**33:B**	**34:C**	**35:A**	**36:D**	**37:B**	**38:C**	**39:A**	**40:D**

Câu 1 – B. Token IDs là $[B, T]$, còn mỗi token được thay bằng vector d phần tử, nên $[B, T, d]$. V là số hàng của bảng tra, không xuất hiện như một chiều trong kết quả lookup. [Slide: 031: 11–14]

Câu 2 – C. ID là nhãn rời rạc; bảng embedding lưu vector học được. Vị trí là tín hiệu riêng, không phải giá trị của ID. [Slide: 031: 8, 12–13]

Câu 3 – A. Bảng token embedding phụ thuộc ID, còn vị trí bổ sung một vector khác qua phép cộng. Vì vậy vector đầu vào có thể khác trong cùng một câu. [Slide: 031: 12–14]

Câu 4 – D. Target dịch trái một token so với input: vị trí chứa a dự đoán b, vị trí chứa d dự đoán e. [Slide: 031: 9–10; 042: 6]

Câu 5 – B. Stride là bước dịch cửa sổ; nó tác động mức độ chồng lắp mẫu, không đổi kiến trúc hay objective. [Slide: 031: 9–10]

Câu 6 – C. Vocabulary tăng làm bảng token embedding cần nhiều hàng và language-model head cần nhiều lựa chọn output. Attention score vẫn phụ thuộc độ dài chuỗi. [Slide: 031: 12; 041: 10–12]

Câu 7 – A. Weights chỉ dùng Q, K; V là phần được tổng hợp có trọng số. Đây là khác biệt giữa chọn nơi chú ý và thông tin được lấy về. [Slide: 032: 9–12]

Câu 8 – C. Điểm attention có shape $[B, H, T, T]$. $d_h = 4$ nằm trong phép nhân $QK^\top$ và bị triệt tiêu ở chiều cuối của score. [Slide: 032: 12–14]

Câu 9 – B. Chia theo $\sqrt{d_k}$ điều tiết độ lớn dot-product; không đổi shape hay bỏ bước chuẩn hóa. [Slide: 032: 9]

Câu 10 – D. Causal mask cho phép nhìn quá khứ và chính token hiện tại, cấm các vị trí tương lai. [Slide: 032: 13]

Câu 11 – **A.** Chỉ hai key đầu hợp lệ và có score bằng nhau; chuẩn hóa softmax trên các giá trị hữu hạn khiến mỗi vị trí được $1/2$. [Slide: 032: 9, 13]

Câu 12 – **B.** Mỗi query có một phân phối mức chú ý trên các key; tổng của một hàng attention bằng 1. Khác softmax LM qua vocabulary. [Slide: 032: 8–9]

Câu 13 – **C.** Bốn phép chiếu vẫn là bốn ma trận $d \times d$ trong giả thiết. Mỗi head hẹp hơn khi số head tăng; tham số không tăng tự động. [Slide: 032: 14]

Câu 14 – **A.** Attention score đếm mọi cặp query–key: $T_{query} \times S_{key}$ trên từng head. [Slide: 032: 6–9]

Câu 15 – **D.** Context c_t phụ thuộc $\alpha_{t,i}$, và weights được tính từ trạng thái decoder cùng hidden states encoder. [Slide: 032: 7–8]

Câu 16 – **A.** Slide dùng sơ đồ encoder BERT và decoder GPT cho hai kiểu objective. Không nên khái quát thành GPT không hiểu ngữ cảnh. [Slide: 031: 3–4]

Câu 17 – **B.** Block của slide là dạng pre-norm: LN1 trước MHA, cộng residual; LN2 trước FFN, cộng residual tiếp. [Slide: 041: 9–10]

Câu 18 – **C.** Thống kê trước affine có thể được chuẩn hóa, nhưng scale/shift học được làm phân phối đầu ra không nhất thiết giữ mean 0. [Slide: 041: 5]

Câu 19 – **D.** Dropout là regularization lúc huấn luyện; inference thường tắt hoạt động ngẫu nhiên này. [Slide: 041: 8]

Câu 20 – **A.** Residual tạo đường truyền gradient/tín hiệu trực tiếp. Bỏ shortcut làm khác tối ưu, nhưng không bắt buộc đổi shape. [Slide: 041: 7–9]

Câu 21 – **B.** Mỗi vị trí của mỗi chuỗi đưa ra một vector 100 logits. Khi generate token kế tiếp chỉ chọn logits của vị trí cuối, không có nghĩa toàn tensor chỉ có hai chiều. [Slide: 041: 4, 12]

Câu 22 – **C.** Mỗi phép chiếu có 64 trọng số; bốn phép chiếu có 256. Không cộng embeddings, FFN hay LayerNorm. [Slide: 032: 14; 041: 9]

Câu 23 – **A.** $-\log p$ giảm khi xác suất token đúng tăng. Chỉ cần tính chất đơn điệu của logarithm. [Slide: 042: 6]

Câu 24 – **D.** Training dùng các cặp input/target dịch một bước với causal mask để song song theo vị trí; inference phụ thuộc output của bước trước. [Slide: 042: 4–7]

Câu 25 – **B.** Validation theo dõi năng lực mô hình, không đóng vai trò batch huấn luyện để cập nhật weights như train. [Slide: 042: 7; 051: 8]

Câu 26 – **C.** Chia cho hằng số dương giữ thứ tự logits. Nhiệt độ lớn làm mềm phân phối, không đổi argmax. [Slide: 042: 8–9]

Câu 27 – **A.** Chỉ còn đúng token top-1 có xác suất khác 0, nên sampling thực tế luôn chọn nó. [Slide: 042: 10]

Câu 28 – **D.** Mask trước softmax khiến xác suất ngoài top-k bằng 0 và xác suất trong top-k được tái chuẩn hóa. [Slide: 042: 10]

Câu 29 – **B.** Điểm cốt lõi của minh họa là thay projection đầu ra cho bài toán classification; không phải biến label thành token vocabulary. [Slide: 051: 6–7]

Câu 30 – **C.** Mô hình với classification head được huấn luyện xuất classes trực tiếp; đây là mục đích phân biệt với prompting language model chung. [Slide: 051: 3–6]

Câu 31 – **A.** Prompt template là cách tổ chức nội dung cho input/output model; task và desired response có thể giữ nguyên. [Slide: 051: 13]

Câu 32 – **D.** Slide ghi rõ mask target tokens khi tính loss; input vẫn chứa instruction để model tạo response có điều kiện. [Slide: 051: 14–15]

Câu 33 – **B.** Không cần huấn luyện model đoán lại chính prompt nếu mục tiêu là assistant response; nhưng instruction vẫn là context. [Slide: 051: 14–15]

Câu 34 – C. Ví dụ trong slide dùng cùng prompt với hai phong cách đúng khác nhau để giải thích tại sao preference learning cần thiết. [Slide: 052: 2–3]

Câu 35 – A. Slide RLHF mô tả SFT làm base policy, human rankings để học reward model, rồi dùng reward cải thiện policy bằng PPO. [Slide: 052: 5–7]

Câu 36 – D. RM học tín hiệu tương quan theo dữ liệu xếp hạng, không phải oracle chân lý tuyệt đối. [Slide: 052: 6–7]

Câu 37 – B. Sơ đồ so sánh cho thấy DPO đi từ preference ranking tới trực tiếp tối ưu LLM policy; công thức vẫn có π_{ref}. [Slide: 052: 9–10]

Câu 38 – C. DPO so sánh policy hiện tại với reference trên cả preferred và dispreferred response, thông qua log-ratios. [Slide: 052: 10]

Câu 39 – A. Slide ghi rõ Llama 3.2 1B có 16 blocks, 2048-d; Qwen3 0.6B có 28 blocks, 1024-d. Cần tách depth và width. [Slide: 062: 3]

Câu 40 – D. Đây là thay đổi cách đưa positional information vào mô hình, không phải bỏ thông tin vị trí. [Slide: 031: 13; 062: 2–3]

18. Đáp án và lời giải – Đề B

1:B	**2:D**	**3:A**	**4:C**	**5:B**	**6:D**	**7:A**	**8:C**	**9:B**	**10:D**
11:A	**12:C**	**13:B**	**14:D**	**15:A**	**16:C**	**17:D**	**18:B**	**19:A**	**20:C**
21:D	**22:B**	**23:C**	**24:A**	**25:D**	**26:B**	**27:A**	**28:C**	**29:D**	**30:C**
31:A	**32:B**	**33:D**	**34:A**	**35:C**	**36:B**	**37:D**	**38:A**	**39:C**	**40:D**

Câu 1 – B. Embedding lookup là truy cập theo chỉ số. Đổi cách đánh số mà đổi hàng theo đúng ánh xạ thì vector tra được vẫn như cũ; lưu ý các thành phần khác dùng ID cũng phải nhất quán. [Slide: 031: 12]

Câu 2 – D. Vì $p_1 \neq p_3$, cộng cùng e_7 cho hai vector vẫn tạo hai vector khác nhau; điều này không nói trước cosine similarity. [Slide: 031: 13–14]

Câu 3 – A. Trục softmax xác định ý nghĩa: attention chọn nguồn thông tin, LM chọn next token, classifier chọn nhãn. [Slide: 032: 9; 041: 12; 051: 6]

Câu 4 – C. Training có sẵn target để đánh giá, nhưng phải hạn chế đường lan truyền thông tin trong forward; causal mask đảm bảo điều này. [Slide: 031: 9–10; 032: 13]

Câu 5 – B. Value tại vị trí có attention weight 0 không góp vào weighted sum cho query ấy; lưu ý phát biểu chỉ nói tác động trực tiếp ở một lần attention. [Slide: 032: 9, 13]

Câu 6 – D. Phân phối trên các key hợp lệ phải cộng bằng 1. Mask trước softmax là cách chuẩn minh họa trong slide. [Slide: 032: 9, 13]

Câu 7 – A. Cho phép các vị trí $j \leq i$, gồm chính nó; với $i = 3$ là 3 vị trí. [Slide: 032: 13]

Câu 8 – C. Bốn heads nhân cặp $T_{decoder} = 2$ queries với $S_{source} = 5$ keys; $[B, H, T, S]$. [Slide: 032: 6–9]

Câu 9 – B. Mỗi head có $d_k = d/H = 4$ theo giả thiết, vì vậy chia $\sqrt{d_k} = 2$. [Slide: 032: 9, 14]

Câu 10 – D. Các heads chia sẻ input nguồn, không bắt buộc chia sẻ projection weights. Đây là ý nghĩa của multi-head. [Slide: 032: 12, 14]

Câu 11 – A. $\sum_i \alpha_i V_i = (\sum_i \alpha_i)v = v$; đây là bài tập suy luận từ công thức weighted sum, không phải phép đo thực nghiệm. [Slide: 032: 8–9]

Câu 12 – C. Số cặp query–key từ T^2 thành $(2T)^2 = 4T^2$. Điều này riêng cho tensor score; không kết luận tổng runtime chính xác gấp 4. [Slide: 032: 9, 14]

Câu 13 – B. Masked self-attention tổng hợp value từ nhiều vị trí. FFN/activation thông thường tác động theo token trên chiều feature. [Slide: 041: 9–10]

Câu 14 – D. Slide diễn đạt "can lead to better optimization properties; không được đổi thành khẳng định tuyệt đối. [Slide: 041: 6]

Câu 15 – A. Nhầm LayerNorm với BatchNorm. Việc chuẩn hóa theo feature cho từng biểu diễn không yêu cầu toàn batch như phát biểu. [Slide: 041: 5]

Câu 16 – C. Ở inference mode dropout tiêu chuẩn tắt; ngẫu nhiên có thể đến từ sampling. Đừng đánh đồng hai nguồn stochasticity. [Slide: 041: 8; 042: 8–10]

Câu 17 – D. Các tensor liên quan trực tiếp tới vocabulary đổi chiều V; attention score tính theo cặp vị trí và số head, không theo V. [Slide: 031: 12; 041: 12]

Câu 18 – B. $4(16)^2/[4(8)^2] = 4$. Khi tăng độ rộng d, ma trận $d \times d$ tăng theo bình phương. [Slide: 032: 14; 041: 11]

Câu 19 – A. Chia logits cho $\tau = 0.5 > 0$ giữ nguyên thứ tự. Greedy lấy argmax ở vị trí đầu. [Slide: 042: 8–9]

Câu 20 – C. $\tau = 1$ không làm thay đổi softmax gốc, nhưng sampling chọn ngẫu nhiên theo phân phối thay vì argmax. [Slide: 042: 8–9]

Câu 21 – D. Hai logits cao nhất là 7 (vị trí 1) và 5 (vị trí 3). Top-k giữ top score, không phải hai token ID nhỏ nhất. [Slide: 042: 10]

Câu 22 – B. Với one-hot target, term loss tại vị trí chỉ phụ thuộc xác suất của ground-truth token. Phân bố trên các token sai có thể khác mà loss term bằng nhau. [Slide: 042: 6]

Câu 23 – C. Có 25/5=5 mini-batches; khi optimizer step mỗi batch thì 5 updates. Đây là bài tính ví dụ theo training loop trên slide. [Slide: 042: 7]

Câu 24 – A. Đây là dấu hiệu kinh điển của overfitting, dù cần thêm dữ liệu để kết luận nguyên nhân chính xác; không dùng validation làm tập update chuẩn. [Slide: 042: 7; 041: 8]

Câu 25 – D. Format thay đổi chuỗi đầu vào có thể đổi token IDs; chính task và desired output không tự đổi. [Slide: 051: 13–14]

Câu 26 – B. Loss mask và attention mask phục vụ hai mục đích khác nhau. Model vẫn condition theo instruction trước khi sinh response. [Slide: 051: 14–15]

Câu 27 – A. Đây là pad masking trên target loss, khác padding attention mask (nếu model còn cần loại mask attention riêng). [Slide: 051: 14]

Câu 28 – C. Classification logits có 2 nhãn; LM logits có V lựa chọn token. Cùng là điểm số/softmax nhưng không cùng không gian kết quả. [Slide: 051: 3–7; 042: 10]

Câu 29 – D. Slide mô tả load pretrained weights rồi modify model cho classification. Có thể freeze/tune các phần khác nhau tùy thiết kế, không phải luật tuyệt đối. [Slide: 051: 5–8]

Câu 30 – C. Margin bằng $\beta \log(R_w/R_l) = \beta \log(1/4) < 0$. Chữ "chosen là nhãn dữ liệu, không ép margin của policy hiện tại phải dương. [Slide: 052: 10]

Câu 31 – A. β nhân chênh lệch log-ratios bên trong sigmoid. Việc thay đổi hiệu quả tối ưu sâu hơn cần giả thiết khác, nhưng đây là vai trò trực tiếp trong công thức. [Slide: 052: 10]

Câu 32 – B. Margin $\beta[\log R_w - \log R_l] = 0$. Điều này thể hiện DPO dùng *chênh lệch* tỷ số so với reference. [Slide: 052: 10]

Câu 33 – D. Tên *direct* có nghĩa bỏ pipeline RM + PPO tách rời, không có nghĩa bỏ reference. Sơ đồ/công thức chứa rõ π_{ref}. [Slide: 052: 9–10]

Câu 34 – A. Sơ đồ chỉ rõ value model $\to v$, cùng reward đi vào GAE rồi tạo advantage A cập nhật policy. [Slide: 052: 11]

Câu 35 – C. GRPO dùng một nhóm samples $o_1, \ldots, o_G$ với rewards để có tín hiệu relative advantage; sơ đồ vẫn có reference/KL. [Slide: 052: 11]

Câu 36 – B. Khi mọi rewards ngang nhau, không có thứ tự hơn/kém tương đối từ reward group. Chi tiết chuẩn hóa cụ thể phụ thuộc implementation. [Slide: 052: 11]

Câu 37 – D. RM học sở thích của dữ liệu quan sát; sở thích và tính đúng đắn là hai tiêu chí có liên quan nhưng không đồng nhất. Đây là suy luận từ mục tiêu RLHF trong slide. [Slide: 052: 3–7]

Câu 38 – A. Weight tying là chia sẻ tham số; model vẫn phải tạo logits trên vocabulary khi sinh văn bản. [Slide: 062: 2]

Câu 39 – C. Sơ đồ Qwen bên phải có nhãn Q/K RMSNorm; hai phía đều có RoPE, grouped-query attention, RMSNorm. [Slide: 062: 3]

Câu 40 – D. Context length là giới hạn vị trí, khác width d, vocab V và số layers. Slide variants cho thấy có thể thay đổi các trục riêng biệt. [Slide: 062: 2–3]

19. Đáp án và lời giải – Đề C (kiểm chứng từ mã nguồn)

Tra R2–R7, RE, RM6 ở phần đầu mở rộng để đọc lại lệnh gốc. Với câu hỏi dùng giả thiết bổ sung, đáp án áp dụng trong đúng phạm vi giả thiết đã nêu.

1:B	**2:D**	**3:C**	**4:A**	**5:C**	**6:B**	**7:D**	**8:A**	**9:B**	**10:C**
11:D	**12:A**	**13:C**	**14:B**	**15:D**	**16:A**	**17:C**	**18:B**	**19:D**	**20:A**
21:C	**22:D**	**23:A**	**24:B**	**25:C**	**26:D**	**27:A**	**28:B**	**29:C**	**30:D**
31:A	**32:B**	**33:C**	**34:D**	**35:A**	**36:B**	**37:C**	**38:D**	**39:A**	**40:B**

Câu 1 – B. `range(0,12-4,3)` sinh các vị trí $0, 3, 6$, tức 3 cặp input–target. Mỗi sample cần cả token kế tiếp cho target. [Code: R2]

Câu 2 – D. Nhắm dự đoán token tiếp theo tại từng vị trí nên target phải dịch đúng một ID: $5 \to 6$, $6 \to 7$, $7 \to 8$. [Code: R2]

Câu 3 – C. Bảng token có $100 \times 8 = 800$, bảng vị trí có $16 \times 8 = 128$, cộng 928. Đây không phải tổng số tham số GPT. [Code: R4]

Câu 4 – A. Embedding là phép tra theo từng ID, thêm chiều vector d; V là kích thước bảng embedding chứ không là chiều kết quả. [Code: R4]

Câu 5 – C. `tok_emb` chỉ phụ thuộc token ID; `pos_emb` phụ thuộc vị trí và được cộng vào cùng không gian d. [Code: R4]

Câu 6 – B. Được nhìn các key ở vị trí $0, 1, 2$ và không được nhìn $3, 4$. Upper-triangular mask chặn tương lai. [Code: R3]

Câu 7 – D. Bước transpose tạo $[B, H, T, d_h]$ với $d_h = 12/3 = 4$; mỗi head nhân $QK^\top$ tạo ma trận $T \times T$. [Code: R3]

Câu 8 – A. Code kiểm tra d_{out} mod $H = 0$ để reshape đúng số kênh; không tự thêm padding. [Code: R3]

Câu 9 – B. Các linear vẫn chiếu $12 \to 12$; chia 12 thành 3 hay 4 nhóm chỉ đổi cách reshape/phân chia channel. [Code: R3]

Câu 10 – C. Softmax tạo tổng hàng 1 trên key hợp lệ; sau đó `self.dropout(attn_weights)` có thể nhân giữ lại theo hệ số và triệt bớt ô. [Code: R3]

Câu 11 – D. Đối số `bias` của các `nn.Linear` Q/K/V là False, nhưng `weight` vẫn là trainable parameters. [Code: R3]

Câu 12 – A. Scores được tính từ $QK^\top$; values chỉ tham gia nhân tổng hợp sau softmax. Đây là điều kiện "giữ Q/K cố định trong cùng phép toán. [Code: R3]

Câu 13 – C. Code lưu shortcut trước, chuẩn hóa x, chạy attention, dropout, rồi cộng shortcut ban đầu. [Code: R4]

Câu 14 – B. Code `mean(dim=-1)` và `var(dim=-1,unbiased=False)` cùng với tham số scale/shift; không chuẩn hóa batch toàn cục. [Code: R4]

Câu 15 – D. Đây là hai điều khiển độc lập; vì vậy code thường kết hợp `eval()` và `no_grad()` khi suy luận/đánh giá. [Code: R4–R5]

Câu 16 – A. `GPTModel` trả $[B, T, V]$; khi autoregressive generate, token kế tiếp theo ngữ cảnh hiện tại nằm ở hàng $T - 1$, còn pretraining giám sát các vị trí đã có target. [Code: R2, R4–R5]

Câu 17 – C. `idx_cond=idx[:, -context_size:]` cắt input forward; `idx` tiếp tục nối các token vừa sinh để làm output. [Code: R5]

Câu 18 – B. Chia logits cho một số dương giữ quan hệ lớn/nhỏ; τ lớn làm chênh lệch logits nhỏ lại và softmax phẳng hơn. [Code: R5]

Câu 19 – D. Các logit dưới ngưỡng top-k thành $-\infty$ trước softmax; với các logit khác nhau, xác suất của 3 token hợp lệ không bằng nhau. [Code: R5]

Câu 20 – A. Tập ứng viên chỉ còn một token; việc làm phẳng xác suất không thể tạo token khác ngoài tập. Điều kiện duy nhất loại trừ trường hợp logit bằng nhau. [Code: R5]

Câu 21 – C. Trước slicing, output $[4, 10, 2]$; code lấy chỉ hàng `[:, -1, :]` để loss so nhãn toàn câu, còn $[4, 2]$. [Code: R6]

Câu 22 – D. Code phân loại dùng chỉ số tensor cuối, không tìm last non-padding token. Lỗi suy luận thường gặp là nhầm `-1` với "token cuối có nghĩa. [Code: R6]

Câu 23 – A. Code explicit mở `trf_blocks[-1]` và `final_norm`; head mới được tạo nên mặc định trainable. [Code: R6]

Câu 24 – B. `create_balanced_dataset` tạo tỷ lệ gần 1:1 trong data dùng train/eval; độ chính xác khi deploy còn phụ thuộc prevalence và dạng sai số. [Code: R6]

Câu 25 – C. Code giữ nguyên lần xuất hiện *đầu tiên* (đóng vai trò kết thúc chuỗi) và mask các padding tiếp theo, tức 2 vị trí. [Code: R7]

Câu 26 – D. `ignore_index` là quy ước của loss, không phải token vocabulary hay causal/padding attention mask. [Code: R7]

Câu 27 – A. Code encode văn bản full instruction–response rồi shift target; chỉ mask padding dư. Mask riêng prompt là lựa chọn mở rộng trong slide chứ không tự diễn ra. [Code: R7]

Câu 28 – B. Adapter có $A \in \mathbb{R}^{12\times 2}$ (24) và $B \in \mathbb{R}^{2\times 8}$ (16), tổng 40. [Code: RE]

Câu 29 – C. Ở code, A khởi tạo kiểu Kaiming, B bằng 0; ban đầu không nhiễu output pretrained, về sau B được cập nhật. [Code: RE]

Câu 30 – D. Hệ số đúng theo code là $\alpha/r = 8/4 = 2$. Không phải mọi cài đặt LoRA đều dùng cùng quy ước scale. [Code: RE]

Câu 31 – A. Vì $N = r(d_{in} + d_{out})$, tham số adapter tỷ lệ tuyến tính theo rank; tác động tối ưu và chất lượng không có tính tuyến tính bảo đảm. [Code: RE]

Câu 32 – B. Code tạo `LoRALayer`/`LinearWithLoRA`, freeze model rồi thay các Linear. QLoRA còn có thành phần quantization base model ngoài mục này. [Code: RE]

Câu 33 – C. Đầu Chương 6 phân biệt compute trong đào tạo và suy luận, nói hai phương pháp có thể bổ trợ nhau. [Code: RM6, 6.1]

Câu 34 – D. Notebook dùng bộ kiểm tra correctness và định dạng; RLVR cần khả năng xác minh, không có nghĩa loại bỏ mọi thiết kế reward. [Code: RM6, 6.1–6.2]

Câu 35 – A. `extract_final_candidate` với `fallback=None` yêu cầu boxed trước khi `grade_answer`; đúng số nhưng sai format vẫn không nhận thưởng ở ví dụ. [Code: RM6, 6.6]

Câu 36 – B. Sampling token rời rạc không cần graph; log-prob của chúng dưới policy hiện tại phải được tính bằng graph để cập nhật model. [Code: RM6, 6.5–6.9]

Câu 37 – C. Giá trị trung bình 0.5; hai reward 1 lớn hơn trung bình nên dương, hai reward 0 nhỏ hơn nên âm. Trong ví dụ `torch.std()` cho khoảng ± 0.866. [Code: RM6, 6.7]

Câu 38 – D. Vì $r_i - \bar{r} = 0$ với mọi i, $A_i = 0$. Đây là nhận định về thành phần policy-gradient notebook; optimizer regularization khác vẫn là vấn đề riêng. [Code: RM6, 6.7–6.10]

Câu 39 – A. Log xác suất chuỗi theo luật nhân xác suất là tổng log xác suất token; mean là trung bình dài-normalized, phù hợp ngữ cảnh scoring khác nhưng không cùng objective PG ở đây. [Code: RM6, 6.8–6.9]

Câu 40 – B. `compute_grpo_loss` đặt `loss=pg_loss` và nói KL sẽ được thêm ở chương sau; không nên đồng nhất bản minh họa này với objective GRPO/PPO đầy đủ. [Code: RM6, 6.10; Slide 052, tr. 11]

20. Bảng ôn tập ngay trước giờ thi

20.1 Nếu chỉ có 15 phút: 6 câu hỏi tự vấn

1. Tôi có phân biệt được $[B, T, d]$, $[B, H, T, T]$, $[B, T, V]$, $[B, C]$ mà không nhầm *head, token, class, vocabulary*?
2. Tôi có giải thích được tại sao V quyết định nội dung output nhưng $QK^\top$ quyết định *trọng số* chọn nội dung?
3. Tôi có thể tự dựng causal mask cho một chuỗi dài 4 và giải thích vì sao mask phải áp trước softmax?
4. Tôi có thể phân biệt teacher forcing, greedy, temperature sampling và top-k chỉ qua một ví dụ logits?
5. Tôi có chỉ ra được khác biệt giữa causal mask, padding attention mask, và label -100?
6. Tôi có phân biệt DPO/chosen–rejected với RLVR/GRPO dùng group advantage, và biết Chương 6 notebook đang dùng phiên bản loss tối giản?

20.2 10 dòng chốt hạ khi gặp câu hỏi bám code

1. Cửa sổ `stride` không đổi mức dịch target: luôn lấy target +1 token trong ví dụ GPTDatasetV1.
2. Causal mask $-\infty$ đặt *trước* softmax; dropout *sau* softmax có thể làm tổng hàng khác 1.
3. d_{out} mod $H = 0$ trong code MHA; tăng heads khi giữ d_{out} không tự tăng weights.
4. `eval()` tắt hành vi dropout, `no_grad()` tắt theo dõi gradient: hai ý khác nhau.
5. LM pretraining dùng targets dịch qua nhiều vị trí; generate chỉ dùng logits vị trí cuối.
6. Classifier Ch. 6 dùng `[:, -1, :]`: không tự chọn last non-PAD.
7. Collate instruction: chỉ PAD targets dư được gán -100; prompt loss không mặc định bị bỏ.
8. LoRA: $A \in \mathbb{R}^{d_{in} \times r}$, $B \in \mathbb{R}^{r \times d_{out}}$, $N = r(d_{in} + d_{out})$, $B = 0$ ban đầu.
9. RLVR: thưởng tự kiểm chứng và format; GRPO: cùng prompt, so sánh rewards trong group.
10. Notebook RL Ch. 6: $A_i = (r_i - \bar{r})/(s_r + \epsilon)$; loss $-\text{mean}(A \cdot \text{sumlogp})$, *chưa* thêm KL trong chương này.

20.3 Chiến thuật đề mở tài liệu giấy

Không tìm câu trả lời theo tên phương pháp trước. Hãy đọc *đối tượng biến đổi* trong đề (tensor, loss, weights, model, preference) rồi tra mục công thức đầu tài liệu. Đánh dấu ngay các phương án tuyệt đối hóa từ như "mọi", "duy nhất", "luôn luôn" và kiểm tra phản ví dụ. Nếu câu tính chỉ hỏi lớn/nhỏ/đổi hay không đổi, dùng *monotonicity, shape và tỷ lệ* thay vì tính log/exponential. Khi phân vân hai đáp án, ưu tiên đáp án có giả thiết được nêu rõ và hệ quả không vượt phạm vi đó.

> **Giới hạn trung thực.** Bản này tổng hợp 7 PDF đã cung cấp và hai nguồn GitHub mà sinh viên chỉ rõ. Phần bổ sung được giới hạn theo các file code/notebook đã đọc. LoRA xuất hiện ở Appendix E; QLoRA không được coi là đã được triển khai đầy đủ trong đó. Các chương khác hoặc bài thực hành ngoài hai repository có thể còn chứa nội dung chưa bao phủ. Các câu luyện chỉ là *giả lập về phong cách tư duy*, không phải câu hỏi do giảng viên ban hành.